Audit trails consist of a series of audit records made up of tokens, which `praudit` prints sequentially, one per line. Each token is of a specific type, such as `header` (an audit record header) or `path` (a file path from a name lookup). The following is an example of an `execve` event:

```
header,133,10,execve(2),0,Mon Sep 25 15:58:03 2006, + 384 msec
exec arg,finger,doug
path,/usr/bin/finger
attribute,555,root,wheel,90,24918,104944
subject,robert,root,wheel,root,wheel,38439,38032,42086,128.232.9.100
return,success,0
trailer,133
```

This audit represents a successful `execve` call, in which the command `finger doug` has been run. The `exec arg` token contains the processed command line presented by the shell to the kernel. The `path` token holds the path to the executable as looked up by the kernel. The `attribute` token describes the binary and includes the file mode. The `subject` token stores the audit user ID, effective user ID and group ID, real user ID and group ID, process ID, session ID, port ID, and login address. Notice that the audit user ID and real user ID differ as the user `robert` switched to the `root` account before running this command, but it is audited using the original authenticated user. The `return` token indicates the successful execution and the `trailer` concludes the record.

XML output format is also supported and can be selected by including `-x`.

Since audit logs may be very large, a subset of records can be selected using `auditreduce`. This example selects all audit records produced for the user `trhodes` stored in AUDITFILE:

```
# auditreduce -u trhodes /var/audit/AUDITFILE | praudit
```

Members of the `audit` group have permission to read audit trails in /var/audit. By default, this group is empty, so only the `root` user can read audit trails. Users may be added to the `audit` group in order to delegate audit review rights. As the ability to track audit log contents provides significant insight into the behavior of users and processes, it is recommended that the delegation of audit review rights be performed with caution.

19.4.1. Live Monitoring Using Audit Pipes

Audit pipes are cloning pseudo-devices which allow applications to tap the live audit record stream. This is primarily of interest to authors of intrusion detection and system monitoring applications. However, the audit pipe device is a convenient way for the administrator to allow live monitoring without running into problems with audit trail file ownership or log rotation interrupting the event stream. To track the live audit event stream:

```
# praudit /dev/auditpipe
```

By default, audit pipe device nodes are accessible only to the `root` user. To make them accessible to the members of the `audit` group, add a `devfs` rule to /etc/devfs.rules:

```
add path 'auditpipe*' mode 0440 group audit
```

See devfs.rules(5) for more information on configuring the devfs file system.



It is easy to produce audit event feedback cycles, in which the viewing of each audit event results in the generation of more audit events. For example, if all network I/O is audited, and `praudit` is run from an SSH session, a continuous stream of audit events will be generated at a high rate, as each event being printed will generate another event. For this reason, it is advisable to run `praudit` on an audit pipe device from sessions without fine-grained I/O auditing.

19.4.2. Rotating and Compressing Audit Trail Files

Audit trails are written to by the kernel and managed by the audit daemon, auditd(8). Administrators should not attempt to use newsyslog.conf(5) or other tools to directly rotate audit logs. Instead, `audit` should be used to shut down auditing, reconfigure the audit system, and perform log rotation. The following command causes the audit daemon to create a new audit log and signal the kernel to switch to using the new log. The old log will be terminated and renamed, at which point it may then be manipulated by the administrator:

```
# audit -n
```

If auditd(8) is not currently running, this command will fail and an error message will be produced.

Adding the following line to /etc/crontab will schedule this rotation every twelve hours:

```
0     */12      *       *       *       root    /usr/sbin/audit -n
```

The change will take effect once /etc/crontab is saved.

Automatic rotation of the audit trail file based on file size is possible using `filesz` in audit_control as described in The audit_control File.

As audit trail files can become very large, it is often desirable to compress or otherwise archive trails once they have been closed by the audit daemon. The audit_warn script can be used to perform customized operations for a variety of audit-related events, including the clean termination of audit trails when they are rotated. For example, the following may be added to /etc/security/audit_warn to compress audit trails on close:

```
#
# Compress audit trail files on close.
#
if [ "$1" = closefile ]; then
        gzip -9 $2
fi
```

Other archiving activities might include copying trail files to a centralized server, deleting old trail files, or reducing the audit trail to remove unneeded records. This script will be run only when audit trail files are cleanly terminated. It will not be run on trails left unterminated following an improper shutdown.

Chapter 20. Storage

20.1. Synopsis

This chapter covers the use of disks and storage media in FreeBSD. This includes SCSI and IDE disks, CD and DVD media, memory-backed disks, and USB storage devices.

Read this chapter to learn:

- How to add additional hard disks to a FreeBSD system.
- How to grow the size of a disk's partition on FreeBSD.
- How to configure FreeBSD to use USB storage devices.
- How to use CD and DVD media on a FreeBSD system.
- How to use the backup programs available under FreeBSD.
- How to set up memory disks.
- What file system snapshots are and how to use them efficiently.
- How to use quotas to limit disk space usage.
- How to encrypt disks and swap to secure them against attackers.
- How to configure a highly available storage network.

Before reading this chapter:

- Know how to configure and install a new FreeBSD kernel.

20.2. Adding Disks

This section describes how to add a new SATA disk to a machine that currently only has a single drive. First, turn off the computer and install the drive in the computer following the instructions of the computer, controller, and drive manufacturers. Reboot the system and become `root`.

Inspect /var/run/dmesg.boot to ensure the new disk was found. In this example, the newly added SATA drive will appear as ada1.

For this example, a single large partition will be created on the new disk. The GPT partitioning scheme will be used in preference to the older and less versatile MBR scheme.



If the disk to be added is not blank, old partition information can be removed with `gpart delete`. See gpart(8) for details.

The partition scheme is created, and then a single partition is added. To improve performance on newer disks with larger hardware block sizes, the partition is aligned to one megabyte boundaries:

```
# gpart create -s GPT ada1
```

```
# gpart add -t freebsd-ufs -a 1M ada1
```

Depending on use, several smaller partitions may be desired. See gpart(8) for options to create partitions smaller than a whole disk.

The disk partition information can be viewed with `gpart show`:

```
% gpart show ada1
=>        34  1465146988  ada1  GPT  (699G)
          34        2014        - free -  (1.0M)
        2048  1465143296     1  freebsd-ufs  (699G)
  1465145344        1678        - free -  (839K)
```

A file system is created in the new partition on the new disk:

```
# newfs -U /dev/ada1p1
```

An empty directory is created as a *mountpoint*, a location for mounting the new disk in the original disk's file system:

```
# mkdir /newdisk
```

Finally, an entry is added to /etc/fstab so the new disk will be mounted automatically at startup:

```
/dev/ada1p1 /newdisk    ufs rw  2   2
```

The new disk can be mounted manually, without restarting the system:

```
# mount /newdisk
```

20.3. Resizing and Growing Disks

A disk's capacity can increase without any changes to the data already present. This happens commonly with virtual machines, when the virtual disk turns out to be too small and is enlarged. Sometimes a disk image is written to a USB memory stick, but does not use the full capacity. Here we describe how to resize or *grow* disk contents to take advantage of increased capacity.

Determine the device name of the disk to be resized by inspecting /var/run/dmesg.boot. In this example, there is only one SATA disk in the system, so the drive will appear as ada0.

List the partitions on the disk to see the current configuration:

```
# gpart show ada0
```

```
=>      34  83886013  ada0  GPT  (48G) [CORRUPT]
        34       128     1  freebsd-boot  (64k)
       162  79691648     2  freebsd-ufs  (38G)
  79691810   4194236     3  freebsd-swap  (2G)
  83886046         1        - free -  (512B)
```



If the disk was formatted with the GPT partitioning scheme, it may show as "corrupted" because the GPT backup partition table is no longer at the end of the drive. Fix the backup partition table with `gpart`:

```
# gpart recover ada0
ada0 recovered
```

Now the additional space on the disk is available for use by a new partition, or an existing partition can be expanded:

```
# gpart show ada0
=>       34  102399933  ada0  GPT  (48G)
         34        128     1  freebsd-boot  (64k)
        162   79691648     2  freebsd-ufs  (38G)
   79691810    4194236     3  freebsd-swap  (2G)
   83886046   18513921        - free -  (8.8G)
```

Partitions can only be resized into contiguous free space. Here, the last partition on the disk is the swap partition, but the second partition is the one that needs to be resized. Swap partitions only contain temporary data, so it can safely be unmounted, deleted, and then recreate the third partition after resizing the second partition.

Disable the swap partition:

```
# swapoff /dev/ada0p3
```

Delete the third partition, specified by the `-i` flag, from the disk *ada0*.

```
# gpart delete -i 3 ada0
ada0p3 deleted
# gpart show ada0
=>       34  102399933  ada0  GPT  (48G)
         34        128     1  freebsd-boot  (64k)
        162   79691648     2  freebsd-ufs  (38G)
   79691810   22708157        - free -  (10G)
```



There is risk of data loss when modifying the partition table of a mounted file system. It is best to perform the following steps on an unmounted file system while

> running from a live CD-ROM or USB device. However, if absolutely necessary, a mounted file system can be resized after disabling GEOM safety features:
>
> ```
> # sysctl kern.geom.debugflags=16
> ```

Resize the partition, leaving room to recreate a swap partition of the desired size. The partition to resize is specified with `-i`, and the new desired size with `-s`. Optionally, alignment of the partition is controlled with `-a`. This only modifies the size of the partition. The file system in the partition will be expanded in a separate step.

```
# gpart resize -i 2 -s 47G -a 4k ada0
ada0p2 resized
# gpart show ada0
=>       34  102399933  ada0  GPT  (48G)
         34        128     1  freebsd-boot  (64k)
        162   98566144     2  freebsd-ufs  (47G)
   98566306    3833661        - free -  (1.8G)
```

Recreate the swap partition and activate it. If no size is specified with `-s`, all remaining space is used:

```
# gpart add -t freebsd-swap -a 4k ada0
ada0p3 added
# gpart show ada0
=>       34  102399933  ada0  GPT  (48G)
         34        128     1  freebsd-boot  (64k)
        162   98566144     2  freebsd-ufs  (47G)
   98566306    3833661     3  freebsd-swap  (1.8G)
# swapon /dev/ada0p3
```

Grow the UFS file system to use the new capacity of the resized partition:

```
# growfs /dev/ada0p2
Device is mounted read-write; resizing will result in temporary write suspension for
/.
It's strongly recommended to make a backup before growing the file system.
OK to grow file system on /dev/ada0p2, mounted on /, from 38GB to 47GB? [Yes/No] Yes
super-block backups (for fsck -b #) at:
 80781312, 82063552, 83345792, 84628032, 85910272, 87192512, 88474752,
 89756992, 91039232, 92321472, 93603712, 94885952, 96168192, 97450432
```

If the file system is ZFS, the resize is triggered by running the `online` subcommand with `-e`:

```
# zpool online -e zroot /dev/ada0p2
```

Both the partition and the file system on it have now been resized to use the newly-available disk space.

20.4. USB Storage Devices

Many external storage solutions, such as hard drives, USB thumbdrives, and CD and DVD burners, use the Universal Serial Bus (USB). FreeBSD provides support for USB 1.x, 2.0, and 3.0 devices.



USB 3.0 support is not compatible with some hardware, including Haswell (Lynx point) chipsets. If FreeBSD boots with a `failed with error 19` message, disable xHCI/USB3 in the system BIOS.

Support for USB storage devices is built into the GENERIC kernel. For a custom kernel, be sure that the following lines are present in the kernel configuration file:

```
device scbus    # SCSI bus (required for ATA/SCSI)
device da   # Direct Access (disks)
device pass # Passthrough device (direct ATA/SCSI access)
device uhci # provides USB 1.x support
device ohci # provides USB 1.x support
device ehci # provides USB 2.0 support
device xhci # provides USB 3.0 support
device usb  # USB Bus (required)
device umass    # Disks/Mass storage - Requires scbus and da
device cd   # needed for CD and DVD burners
```

FreeBSD uses the umass(4) driver which uses the SCSI subsystem to access USB storage devices. Since any USB device will be seen as a SCSI device by the system, if the USB device is a CD or DVD burner, do *not* include `device atapicam` in a custom kernel configuration file.

The rest of this section demonstrates how to verify that a USB storage device is recognized by FreeBSD and how to configure the device so that it can be used.

20.4.1. Device Configuration

To test the USB configuration, plug in the USB device. Use `dmesg` to confirm that the drive appears in the system message buffer. It should look something like this:

```
umass0: <STECH Simple Drive, class 0/0, rev 2.00/1.04, addr 3> on usbus0
umass0:  SCSI over Bulk-Only; quirks = 0x0100
umass0:4:0:-1: Attached to scbus4
da0 at umass-sim0 bus 0 scbus4 target 0 lun 0
da0: <STECH Simple Drive 1.04> Fixed Direct Access SCSI-4 device
da0: Serial Number WD-WXE508CAN263
da0: 40.000MB/s transfers
da0: 152627MB (312581808 512 byte sectors: 255H 63S/T 19457C)
da0: quirks=0x2<NO_6_BYTE>
```

The brand, device node (da0), speed, and size will differ according to the device.

Since the USB device is seen as a SCSI one, `camcontrol` can be used to list the USB storage devices attached to the system:

```
# camcontrol devlist
<STECH Simple Drive 1.04>          at scbus4 target 0 lun 0 (pass3,da0)
```

Alternately, `usbconfig` can be used to list the device. Refer to usbconfig(8) for more information about this command.

```
# usbconfig
ugen0.3: <Simple Drive STECH> at usbus0, cfg=0 md=HOST spd=HIGH (480Mbps) pwr=ON (2mA)
```

If the device has not been formatted, refer to Adding Disks for instructions on how to format and create partitions on the USB drive. If the drive comes with a file system, it can be mounted by `root` using the instructions in "Mounting and Unmounting File Systems".



Allowing untrusted users to mount arbitrary media, by enabling `vfs.usermount` as described below, should not be considered safe from a security point of view. Most file systems were not built to safeguard against malicious devices.

To make the device mountable as a normal user, one solution is to make all users of the device a member of the `operator` group using pw(8). Next, ensure that `operator` is able to read and write the device by adding these lines to /etc/devfs.rules:

```
[localrules=5]
add path 'da*' mode 0660 group operator
```



If internal SCSI disks are also installed in the system, change the second line as follows:

```
add path 'da[3-9]*' mode 0660 group operator
```

This will exclude the first three SCSI disks (da0 to da2) from belonging to the `operator` group. Replace *3* with the number of internal SCSI disks. Refer to devfs.rules(5) for more information about this file.

Next, enable the ruleset in /etc/rc.conf:

```
devfs_system_ruleset="localrules"
```

Then, instruct the system to allow regular users to mount file systems by adding the following line

to /etc/sysctl.conf:

```
vfs.usermount=1
```

Since this only takes effect after the next reboot, use `sysctl` to set this variable now:

```
# sysctl vfs.usermount=1
vfs.usermount: 0 -> 1
```

The final step is to create a directory where the file system is to be mounted. This directory needs to be owned by the user that is to mount the file system. One way to do that is for `root` to create a subdirectory owned by that user as /mnt/username. In the following example, replace *username* with the login name of the user and *usergroup* with the user's primary group:

```
# mkdir /mnt/username
# chown username:usergroup /mnt/username
```

Suppose a USB thumbdrive is plugged in, and a device /dev/da0s1 appears. If the device is formatted with a FAT file system, the user can mount it using:

```
% mount -t msdosfs -o -m=644,-M=755 /dev/da0s1 /mnt/username
```

Before the device can be unplugged, it *must* be unmounted first:

```
% umount /mnt/username
```

After device removal, the system message buffer will show messages similar to the following:

```
umass0: at uhub3, port 2, addr 3 (disconnected)
da0 at umass-sim0 bus 0 scbus4 target 0 lun 0
da0: <STECH Simple Drive 1.04> s/n WD-WXE508CAN263          detached
(da0:umass-sim0:0:0:0): Periph destroyed
```

20.4.2. Automounting Removable Media

USB devices can be automatically mounted by uncommenting this line in /etc/auto_master:

```
/media      -media      -nosuid
```

Then add these lines to /etc/devd.conf:

```
notify 100 {
    match "system" "GEOM";
    match "subsystem" "DEV";
    action "/usr/sbin/automount -c";
};
```

Reload the configuration if autofs(5) and devd(8) are already running:

```
# service automount restart
# service devd restart
```

autofs(5) can be set to start at boot by adding this line to /etc/rc.conf:

```
autofs_enable="YES"
```

autofs(5) requires devd(8) to be enabled, as it is by default.

Start the services immediately with:

```
# service automount start
# service automountd start
# service autounmountd start
# service devd start
```

Each file system that can be automatically mounted appears as a directory in /media/. The directory is named after the file system label. If the label is missing, the directory is named after the device node.

The file system is transparently mounted on the first access, and unmounted after a period of inactivity. Automounted drives can also be unmounted manually:

```
# automount -fu
```

This mechanism is typically used for memory cards and USB memory sticks. It can be used with any block device, including optical drives or iSCSILUNs.

20.5. Creating and Using CD Media

Compact Disc (CD) media provide a number of features that differentiate them from conventional disks. They are designed so that they can be read continuously without delays to move the head between tracks. While CD media do have tracks, these refer to a section of data to be read continuously, and not a physical property of the disk. The ISO 9660 file system was designed to deal with these differences.

The FreeBSD Ports Collection provides several utilities for burning and duplicating audio and data CDs. This chapter demonstrates the use of several command line utilities. For CD burning software with a graphical utility, consider installing the sysutils/xcdroast or sysutils/k3b packages or ports.

20.5.1. Supported Devices

The GENERIC kernel provides support for SCSI, USB, and ATAPICD readers and burners. If a custom kernel is used, the options that need to be present in the kernel configuration file vary by the type of device.

For a SCSI burner, make sure these options are present:

```
device scbus    # SCSI bus (required for ATA/SCSI)
device da   # Direct Access (disks)
device pass # Passthrough device (direct ATA/SCSI access)
device cd   # needed for CD and DVD burners
```

For a USB burner, make sure these options are present:

```
device scbus    # SCSI bus (required for ATA/SCSI)
device da   # Direct Access (disks)
device pass # Passthrough device (direct ATA/SCSI access)
device cd   # needed for CD and DVD burners
device uhci # provides USB 1.x support
device ohci # provides USB 1.x support
device ehci # provides USB 2.0 support
device xhci # provides USB 3.0 support
device usb  # USB Bus (required)
device umass    # Disks/Mass storage - Requires scbus and da
```

For an ATAPI burner, make sure these options are present:

```
device ata  # Legacy ATA/SATA controllers
device scbus    # SCSI bus (required for ATA/SCSI)
device pass # Passthrough device (direct ATA/SCSI access)
device cd   # needed for CD and DVD burners
```



On FreeBSD versions prior to 10.x, this line is also needed in the kernel configuration file if the burner is an ATAPI device:

```
device atapicam
```

Alternately, this driver can be loaded at boot time by adding the following line to /boot/loader.conf:

```
atapicam_load="YES"
```

This will require a reboot of the system as this driver can only be loaded at boot time.

To verify that FreeBSD recognizes the device, run `dmesg` and look for an entry for the device. On systems prior to 10.x, the device name in the first line of the output will be acd0 instead of cd0.

```
% dmesg | grep cd
cd0 at ahcich1 bus 0 scbus1 target 0 lun 0
cd0: <HL-DT-ST DVDRAM GU70N LT20> Removable CD-ROM SCSI-0 device
cd0: Serial Number M3OD3S34152
cd0: 150.000MB/s transfers (SATA 1.x, UDMA6, ATAPI 12bytes, PIO 8192bytes)
cd0: Attempt to query device size failed: NOT READY, Medium not present - tray closed
```

20.5.2. Burning a CD

In FreeBSD, `cdrecord` can be used to burn CDs. This command is installed with the sysutils/cdrtools package or port.

While `cdrecord` has many options, basic usage is simple. Specify the name of the ISO file to burn and, if the system has multiple burner devices, specify the name of the device to use:

```
# cdrecord dev=device imagefile.iso
```

To determine the device name of the burner, use `-scanbus` which might produce results like this:

```
# cdrecord -scanbus
ProDVD-ProBD-Clone 3.00 (amd64-unknown-freebsd10.0) Copyright (C) 1995-2010 Jörg
Schilling
Using libscg version 'schily-0.9'
scsibus0:
        0,0,0     0) 'SEAGATE ' 'ST39236LW       ' '0004' Disk
        0,1,0     1) 'SEAGATE ' 'ST39173W        ' '5958' Disk
        0,2,0     2) *
        0,3,0     3) 'iomega  ' 'jaz 1GB         ' 'J.86' Removable Disk
        0,4,0     4) 'NEC     ' 'CD-ROM DRIVE:466' '1.26' Removable CD-ROM
        0,5,0     5) *
        0,6,0     6) *
        0,7,0     7) *
scsibus1:
        1,0,0   100) *
        1,1,0   101) *
        1,2,0   102) *
        1,3,0   103) *
        1,4,0   104) *
```

```
        1,5,0   105) 'YAMAHA  ' 'CRW4260         ' '1.0q' Removable CD-ROM
        1,6,0   106) 'ARTEC   ' 'AM12S          ' '1.06' Scanner
        1,7,0   107) *
```

Locate the entry for the CD burner and use the three numbers separated by commas as the value for `dev`. In this case, the Yamaha burner device is `1,5,0`, so the appropriate input to specify that device is `dev=1,5,0`. Refer to the manual page for `cdrecord` for other ways to specify this value and for information on writing audio tracks and controlling the write speed.

Alternately, run the following command to get the device address of the burner:

```
# camcontrol devlist
<MATSHITA CDRW/DVD UJDA740 1.00>   at scbus1 target 0 lun 0 (cd0,pass0)
```

Use the numeric values for `scbus`, `target`, and `lun`. For this example, `1,0,0` is the device name to use.

20.5.3. Writing Data to an ISO File System

In order to produce a data CD, the data files that are going to make up the tracks on the CD must be prepared before they can be burned to the CD. In FreeBSD, sysutils/cdrtools installs `mkisofs`, which can be used to produce an ISO 9660 file system that is an image of a directory tree within a UNIX® file system. The simplest usage is to specify the name of the ISO file to create and the path to the files to place into the ISO 9660 file system:

```
# mkisofs -o imagefile.iso /path/to/tree
```

This command maps the file names in the specified path to names that fit the limitations of the standard ISO 9660 file system, and will exclude files that do not meet the standard for ISO file systems.

A number of options are available to overcome the restrictions imposed by the standard. In particular, `-R` enables the Rock Ridge extensions common to UNIX® systems and `-J` enables Joliet extensions used by Microsoft® systems.

For CDs that are going to be used only on FreeBSD systems, `-U` can be used to disable all filename restrictions. When used with `-R`, it produces a file system image that is identical to the specified FreeBSD tree, even if it violates the ISO 9660 standard.

The last option of general use is `-b`. This is used to specify the location of a boot image for use in producing an "El Torito" bootable CD. This option takes an argument which is the path to a boot image from the top of the tree being written to the CD. By default, `mkisofs` creates an ISO image in "floppy disk emulation" mode, and thus expects the boot image to be exactly 1200, 1440 or 2880 KB in size. Some boot loaders, like the one used by the FreeBSD distribution media, do not use emulation mode. In this case, `-no-emul-boot` should be used. So, if /tmp/myboot holds a bootable FreeBSD system with the boot image in /tmp/myboot/boot/cdboot, this command would produce /tmp/bootable.iso:

```
# mkisofs -R -no-emul-boot -b boot/cdboot -o /tmp/bootable.iso /tmp/myboot
```

The resulting ISO image can be mounted as a memory disk with:

```
# mdconfig -a -t vnode -f /tmp/bootable.iso -u 0
# mount -t cd9660 /dev/md0 /mnt
```

One can then verify that /mnt and /tmp/myboot are identical.

There are many other options available for `mkisofs` to fine-tune its behavior. Refer to mkisofs(8) for details.



It is possible to copy a data CD to an image file that is functionally equivalent to the image file created with `mkisofs`. To do so, use dd with the device name as the input file and the name of the ISO to create as the output file:

```
# dd if=/dev/cd0 of=file.iso bs=2048
```

The resulting image file can be burned to CD as described in Burning a CD.

20.5.4. Using Data CDs

Once an ISO has been burned to a CD, it can be mounted by specifying the file system type, the name of the device containing the CD, and an existing mount point:

```
# mount -t cd9660 /dev/cd0 /mnt
```

Since `mount` assumes that a file system is of type `ufs`, an `Incorrect super block` error will occur if `-t cd9660` is not included when mounting a data CD.

While any data CD can be mounted this way, disks with certain ISO 9660 extensions might behave oddly. For example, Joliet disks store all filenames in two-byte Unicode characters. If some non-English characters show up as question marks, specify the local charset with `-C`. For more information, refer to mount_cd9660(8).



In order to do this character conversion with the help of `-C`, the kernel requires the cd9660_iconv.ko module to be loaded. This can be done either by adding this line to loader.conf:

```
cd9660_iconv_load="YES"
```

and then rebooting the machine, or by directly loading the module with `kldload`.

Occasionally, `Device not configured` will be displayed when trying to mount a data CD. This usually means that the CD drive has not detected a disk in the tray, or that the drive is not visible on the bus. It can take a couple of seconds for a CD drive to detect media, so be patient.

Sometimes, a SCSICD drive may be missed because it did not have enough time to answer the bus reset. To resolve this, a custom kernel can be created which increases the default SCSI delay. Add the following option to the custom kernel configuration file and rebuild the kernel using the instructions in "Building and Installing a Custom Kernel":

```
options SCSI_DELAY=15000
```

This tells the SCSI bus to pause 15 seconds during boot, to give the CD drive every possible chance to answer the bus reset.



It is possible to burn a file directly to CD, without creating an ISO 9660 file system. This is known as burning a raw data CD and some people do this for backup purposes.

This type of disk can not be mounted as a normal data CD. In order to retrieve the data burned to such a CD, the data must be read from the raw device node. For example, this command will extract a compressed tar file located on the second CD device into the current working directory:

```
# tar xzvf /dev/cd1
```

In order to mount a data CD, the data must be written using `mkisofs`.

20.5.5. Duplicating Audio CDs

To duplicate an audio CD, extract the audio data from the CD to a series of files, then write these files to a blank CD.

Duplicating an Audio CD describes how to duplicate and burn an audio CD. If the FreeBSD version is less than 10.0 and the device is ATAPI, the `atapicam` module must be first loaded using the instructions in Supported Devices.

Procedure: Duplicating an Audio CD

1. The sysutils/cdrtools package or port installs `cdda2wav`. This command can be used to extract all of the audio tracks, with each track written to a separate WAV file in the current working directory:

   ```
   % cdda2wav -vall -B -Owav
   ```

   A device name does not need to be specified if there is only one CD device on the system. Refer to the `cdda2wav` manual page for instructions on how to specify a device and to learn more about the other options available for this command.

2. Use `cdrecord` to write the .wav files:

```
% cdrecord -v dev=2,0 -dao -useinfo  *.wav
```

Make sure that *2,0* is set appropriately, as described in Burning a CD.

20.6. Creating and Using DVD Media

Compared to the CD, the DVD is the next generation of optical media storage technology. The DVD can hold more data than any CD and is the standard for video publishing.

Five physical recordable formats can be defined for a recordable DVD:

- DVD-R: This was the first DVD recordable format available. The DVD-R standard is defined by the DVD Forum. This format is write once.
- DVD-RW: This is the rewritable version of the DVD-R standard. A DVD-RW can be rewritten about 1000 times.
- DVD-RAM: This is a rewritable format which can be seen as a removable hard drive. However, this media is not compatible with most DVD-ROM drives and DVD-Video players as only a few DVD writers support the DVD-RAM format. Refer to Using a DVD-RAM for more information on DVD-RAM use.
- DVD+RW: This is a rewritable format defined by the DVD+RW Alliance. A DVD+RW can be rewritten about 1000 times.
- DVD+R: This format is the write once variation of the DVD+RW format.

A single layer recordable DVD can hold up to 4,700,000,000 bytes which is actually 4.38 GB or 4485 MB as 1 kilobyte is 1024 bytes.



A distinction must be made between the physical media and the application. For example, a DVD-Video is a specific file layout that can be written on any recordable DVD physical media such as DVD-R, DVD+R, or DVD-RW. Before choosing the type of media, ensure that both the burner and the DVD-Video player are compatible with the media under consideration.

20.6.1. Configuration

To perform DVD recording, use growisofs(1). This command is part of the sysutils/dvd+rw-tools utilities which support all DVD media types.

These tools use the SCSI subsystem to access the devices, therefore ATAPI/CAM support must be loaded or statically compiled into the kernel. This support is not needed if the burner uses the USB interface. Refer to USB Storage Devices for more details on USB device configuration.

DMA access must also be enabled for ATAPI devices, by adding the following line to /boot/loader.conf:

```
hw.ata.atapi_dma="1"
```

Before attempting to use dvd+rw-tools, consult the Hardware Compatibility Notes.



For a graphical user interface, consider using sysutils/k3b which provides a user friendly interface to growisofs(1) and many other burning tools.

20.6.2. Burning Data DVDs

Since growisofs(1) is a front-end to mkisofs, it will invoke mkisofs(8) to create the file system layout and perform the write on the DVD. This means that an image of the data does not need to be created before the burning process.

To burn to a DVD+R or a DVD-R the data in /path/to/data, use the following command:

```
# growisofs -dvd-compat -Z /dev/cd0 -J -R /path/to/data
```

In this example, `-J -R` is passed to mkisofs(8) to create an ISO 9660 file system with Joliet and Rock Ridge extensions. Refer to mkisofs(8) for more details.

For the initial session recording, `-Z` is used for both single and multiple sessions. Replace */dev/cd0*, with the name of the DVD device. Using `-dvd-compat` indicates that the disk will be closed and that the recording will be unappendable. This should also provide better media compatibility with DVD-ROM drives.

To burn a pre-mastered image, such as *imagefile.iso*, use:

```
# growisofs -dvd-compat -Z /dev/cd0=imagefile.iso
```

The write speed should be detected and automatically set according to the media and the drive being used. To force the write speed, use `-speed=`. Refer to growisofs(1) for example usage.



In order to support working files larger than 4.38GB, an UDF/ISO-9660 hybrid file system must be created by passing `-udf -iso-level 3` to mkisofs(8) and all related programs, such as growisofs(1). This is required only when creating an ISO image file or when writing files directly to a disk. Since a disk created this way must be mounted as an UDF file system with mount_udf(8), it will be usable only on an UDF aware operating system. Otherwise it will look as if it contains corrupted files.

To create this type of ISO file:

```
% mkisofs -R -J -udf -iso-level 3 -o imagefile.iso /path/to/data
```

To burn files directly to a disk:

```
# growisofs -dvd-compat -udf -iso-level 3 -Z /dev/cd0 -J -R
/path/to/data
```

When an ISO image already contains large files, no additional options are required for growisofs(1) to burn that image on a disk.

Be sure to use an up-to-date version of sysutils/cdrtools, which contains mkisofs(8), as an older version may not contain large files support. If the latest version does not work, install sysutils/cdrtools-devel and read its mkisofs(8).

20.6.3. Burning a DVD-Video

A DVD-Video is a specific file layout based on the ISO 9660 and micro-UDF (M-UDF) specifications. Since DVD-Video presents a specific data structure hierarchy, a particular program such as multimedia/dvdauthor is needed to author the DVD.

If an image of the DVD-Video file system already exists, it can be burned in the same way as any other image. If `dvdauthor` was used to make the DVD and the result is in /path/to/video, the following command should be used to burn the DVD-Video:

```
# growisofs -Z /dev/cd0 -dvd-video /path/to/video
```

`-dvd-video` is passed to mkisofs(8) to instruct it to create a DVD-Video file system layout. This option implies the `-dvd-compat` growisofs(1) option.

20.6.4. Using a DVD+RW

Unlike CD-RW, a virgin DVD+RW needs to be formatted before first use. It is *recommended* to let growisofs(1) take care of this automatically whenever appropriate. However, it is possible to use `dvd+rw-format` to format the DVD+RW:

```
# dvd+rw-format /dev/cd0
```

Only perform this operation once and keep in mind that only virgin DVD+RW medias need to be formatted. Once formatted, the DVD+RW can be burned as usual.

To burn a totally new file system and not just append some data onto a DVD+RW, the media does not need to be blanked first. Instead, write over the previous recording like this:

```
# growisofs -Z /dev/cd0 -J -R /path/to/newdata
```

The DVD+RW format supports appending data to a previous recording. This operation consists of merging a new session to the existing one as it is not considered to be multi-session writing. growisofs(1) will *grow* the ISO 9660 file system present on the media.

For example, to append data to a DVD+RW, use the following:

```
# growisofs -M /dev/cd0 -J -R /path/to/nextdata
```

The same mkisofs(8) options used to burn the initial session should be used during next writes.



Use `-dvd-compat` for better media compatibility with DVD-ROM drives. When using DVD+RW, this option will not prevent the addition of data.

To blank the media, use:

```
# growisofs -Z /dev/cd0=/dev/zero
```

20.6.5. Using a DVD-RW

A DVD-RW accepts two disc formats: incremental sequential and restricted overwrite. By default, DVD-RW discs are in sequential format.

A virgin DVD-RW can be directly written without being formatted. However, a non-virgin DVD-RW in sequential format needs to be blanked before writing a new initial session.

To blank a DVD-RW in sequential mode:

```
# dvd+rw-format -blank=full /dev/cd0
```



A full blanking using `-blank=full` will take about one hour on a 1x media. A fast blanking can be performed using `-blank`, if the DVD-RW will be recorded in Disk-At-Once (DAO) mode. To burn the DVD-RW in DAO mode, use the command:

```
# growisofs -use-the-force-luke=dao -Z /dev/cd0=imagefile.iso
```

Since growisofs(1) automatically attempts to detect fast blanked media and engage DAO write, `-use-the-force-luke=dao` should not be required.

One should instead use restricted overwrite mode with any DVD-RW as this format is more flexible than the default of incremental sequential.

To write data on a sequential DVD-RW, use the same instructions as for the other DVD formats:

```
# growisofs -Z /dev/cd0 -J -R /path/to/data
```

To append some data to a previous recording, use `-M` with growisofs(1). However, if data is appended on a DVD-RW in incremental sequential mode, a new session will be created on the disc and the result will be a multi-session disc.

A DVD-RW in restricted overwrite format does not need to be blanked before a new initial session. Instead, overwrite the disc with `-Z`. It is also possible to grow an existing ISO 9660 file system written on the disc with `-M`. The result will be a one-session DVD.

To put a DVD-RW in restricted overwrite format, the following command must be used:

```
# dvd+rw-format /dev/cd0
```

To change back to sequential format, use:

```
# dvd+rw-format -blank=full /dev/cd0
```

20.6.6. Multi-Session

Few DVD-ROM drives support multi-session DVDs and most of the time only read the first session. DVD+R, DVD-R and DVD-RW in sequential format can accept multiple sessions. The notion of multiple sessions does not exist for the DVD+RW and the DVD-RW restricted overwrite formats.

Using the following command after an initial non-closed session on a DVD+R, DVD-R, or DVD-RW in sequential format, will add a new session to the disc:

```
# growisofs -M /dev/cd0 -J -R /path/to/nextdata
```

Using this command with a DVD+RW or a DVD-RW in restricted overwrite mode will append data while merging the new session to the existing one. The result will be a single-session disc. Use this method to add data after an initial write on these types of media.



Since some space on the media is used between each session to mark the end and start of sessions, one should add sessions with a large amount of data to optimize media space. The number of sessions is limited to 154 for a DVD+R, about 2000 for a DVD-R, and 127 for a DVD+R Double Layer.

20.6.7. For More Information

To obtain more information about a DVD, use `dvd+rw-mediainfo /dev/cd0` while the disc in the specified drive.

More information about dvd+rw-tools can be found in growisofs(1), on the dvd+rw-tools web site, and in the cdwrite mailing list archives.



When creating a problem report related to the use of dvd+rw-tools, always include the output of `dvd+rw-mediainfo`.

20.6.8. Using a DVD-RAM

DVD-RAM writers can use either a SCSI or ATAPI interface. For ATAPI devices, DMA access has to be enabled by adding the following line to /boot/loader.conf:

```
hw.ata.atapi_dma="1"
```

A DVD-RAM can be seen as a removable hard drive. Like any other hard drive, the DVD-RAM must be formatted before it can be used. In this example, the whole disk space will be formatted with a standard UFS2 file system:

```
# dd if=/dev/zero of=/dev/acd0 bs=2k count=1
# bsdlabel -Bw acd0
# newfs /dev/acd0
```

The DVD device, acd0, must be changed according to the configuration.

Once the DVD-RAM has been formatted, it can be mounted as a normal hard drive:

```
# mount /dev/acd0 /mnt
```

Once mounted, the DVD-RAM will be both readable and writeable.

20.7. Creating and Using Floppy Disks

This section explains how to format a 3.5 inch floppy disk in FreeBSD.

Procedure: Steps to Format a Floppy

A floppy disk needs to be low-level formatted before it can be used. This is usually done by the vendor, but formatting is a good way to check media integrity. To low-level format the floppy disk on FreeBSD, use fdformat(1). When using this utility, make note of any error messages, as these can help determine if the disk is good or bad.

1. To format the floppy, insert a new 3.5 inch floppy disk into the first floppy drive and issue:

   ```
   # /usr/sbin/fdformat -f 1440 /dev/fd0
   ```

2. After low-level formatting the disk, create a disk label as it is needed by the system to determine the size of the disk and its geometry. The supported geometry values are listed in /etc/disktab.

   To write the disk label, use bsdlabel(8):

```
# /sbin/bsdlabel -B -w /dev/fd0 fd1440
```

3. The floppy is now ready to be high-level formatted with a file system. The floppy's file system can be either UFS or FAT, where FAT is generally a better choice for floppies.

   To format the floppy with FAT, issue:

   ```
   # /sbin/newfs_msdos /dev/fd0
   ```

The disk is now ready for use. To use the floppy, mount it with mount_msdosfs(8). One can also install and use emulators/mtools from the Ports Collection.

20.8. Backup Basics

Implementing a backup plan is essential in order to have the ability to recover from disk failure, accidental file deletion, random file corruption, or complete machine destruction, including destruction of on-site backups.

The backup type and schedule will vary, depending upon the importance of the data, the granularity needed for file restores, and the amount of acceptable downtime. Some possible backup techniques include:

- Archives of the whole system, backed up onto permanent, off-site media. This provides protection against all of the problems listed above, but is slow and inconvenient to restore from, especially for non-privileged users.
- File system snapshots, which are useful for restoring deleted files or previous versions of files.
- Copies of whole file systems or disks which are synchronized with another system on the network using a scheduled net/rsync.
- Hardware or software RAID, which minimizes or avoids downtime when a disk fails.

Typically, a mix of backup techniques is used. For example, one could create a schedule to automate a weekly, full system backup that is stored off-site and to supplement this backup with hourly ZFS snapshots. In addition, one could make a manual backup of individual directories or files before making file edits or deletions.

This section describes some of the utilities which can be used to create and manage backups on a FreeBSD system.

20.8.1. File System Backups

The traditional UNIX® programs for backing up a file system are dump(8), which creates the backup, and restore(8), which restores the backup. These utilities work at the disk block level, below the abstractions of the files, links, and directories that are created by file systems. Unlike other backup software, `dump` backs up an entire file system and is unable to backup only part of a file system or a directory tree that spans multiple file systems. Instead of writing files and

directories, dump writes the raw data blocks that comprise files and directories.



If dump is used on the root directory, it will not back up /home, /usr, or many other directories since these are typically mount points for other file systems or symbolic links into those file systems.

When used to restore data, restore stores temporary files in /tmp/ by default. When using a recovery disk with a small /tmp, set TMPDIR to a directory with more free space for the restore to succeed.

When using dump, be aware that some quirks remain from its early days in Version 6 of AT&T UNIX®,circa 1975. The default parameters assume a backup to a 9-track tape, rather than to another type of media or to the high-density tapes available today. These defaults must be overridden on the command line.

It is possible to backup a file system across the network to another system or a tape drive attached to another computer. While the rdump(8) and rrestore(8) utilities can be used for this purpose, they are not considered to be secure.

Instead, one can use dump and restore more securely over an SSH connection. This example creates a full, compressed backup of /usr and sends the backup file to the specified host over an SSH connection.

Example 26. Using dump over ssh

```
# /sbin/dump -0uan -f - /usr | gzip -2 | ssh -c blowfish \
          targetuser@targetmachine.example.com dd of=/mybigfiles/dump-usr-l0.gz
```

This example sets RSH in order to write the backup to a tape drive on a remote system over an SSH connection:

Example 27. Using dump over ssh with RSH Set

```
# env RSH=/usr/bin/ssh /sbin/dump -0uan -f
targetuser@targetmachine.example.com:/dev/sa0 /usr
```



Systems using the Z file system (ZFS) can make use of zfs(8) for creating snapshots, as well as sending and receiving them to/from remote systems.

20.8.2. Directory Backups

Several built-in utilities are available for backing up and restoring specified files and directories as needed.

A good choice for making a backup of all of the files in a directory is tar(1). This utility dates back to

Version 6 of AT&T UNIX® and by default assumes a recursive backup to a local tape device. Switches can be used to instead specify the name of a backup file.

This example creates a compressed backup of the current directory and saves it to /tmp/mybackup.tgz. When creating a backup file, make sure that the backup is not saved to the same directory that is being backed up.

Example 28. Backing Up the Current Directory with `tar`

```
# tar czvf /tmp/mybackup.tgz .
```

To restore the entire backup, `cd` into the directory to restore into and specify the name of the backup. Note that this will overwrite any newer versions of files in the restore directory. When in doubt, restore to a temporary directory or specify the name of the file within the backup to restore.

Example 29. Restoring Up the Current Directory with `tar`

```
# tar xzvf /tmp/mybackup.tgz
```

There are dozens of available switches which are described in tar(1). This utility also supports the use of exclude patterns to specify which files should not be included when backing up the specified directory or restoring files from a backup.

To create a backup using a specified list of files and directories, cpio(1) is a good choice. Unlike `tar`, `cpio` does not know how to walk the directory tree and it must be provided the list of files to backup.

For example, a list of files can be created using `ls` or `find`. This example creates a recursive listing of the current directory which is then piped to `cpio` in order to create an output backup file named /tmp/mybackup.cpio.

Example 30. Using `ls` and `cpio` to Make a Recursive Backup of the Current Directory

```
# ls -R | cpio -ovF /tmp/mybackup.cpio
```

A backup utility which tries to bridge the features provided by `tar` and `cpio` is pax(1). Over the years, the various versions of `tar` and `cpio` became slightly incompatible. POSIX® created `pax` which attempts to read and write many of the various `cpio` and `tar` formats, plus new formats of its own.

The `pax` equivalent to the previous examples would be:

Example 31. Backing Up the Current Directory with `pax`

```
# pax -wf /tmp/mybackup.pax .
```

20.8.3. Using Data Tapes for Backups

While tape technology has continued to evolve, modern backup systems tend to combine off-site backups with local removable media. FreeBSD supports any tape drive that uses SCSI, such as LTO or DAT. There is limited support for SATA and USB tape drives.

For SCSI tape devices, FreeBSD uses the sa(4) driver and the /dev/sa0, /dev/nsa0, and /dev/esa0 devices. The physical device name is /dev/sa0. When /dev/nsa0 is used, the backup application will not rewind the tape after writing a file, which allows writing more than one file to a tape. Using /dev/esa0 ejects the tape after the device is closed.

In FreeBSD, `mt` is used to control operations of the tape drive, such as seeking through files on a tape or writing tape control marks to the tape. For example, the first three files on a tape can be preserved by skipping past them before writing a new file:

```
# mt -f /dev/nsa0 fsf 3
```

This utility supports many operations. Refer to mt(1) for details.

To write a single file to tape using `tar`, specify the name of the tape device and the file to backup:

```
# tar cvf /dev/sa0 file
```

To recover files from a `tar` archive on tape into the current directory:

```
# tar xvf /dev/sa0
```

To backup a UFS file system, use `dump`. This examples backs up /usr without rewinding the tape when finished:

```
# dump -0aL -b64 -f /dev/nsa0 /usr
```

To interactively restore files from a `dump` file on tape into the current directory:

```
# restore -i -f /dev/nsa0
```

20.8.4. Third-Party Backup Utilities

The FreeBSD Ports Collection provides many third-party utilities which can be used to schedule the creation of backups, simplify tape backup, and make backups easier and more convenient. Many of these applications are client/server based and can be used to automate the backups of a single system or all of the computers in a network.

Popular utilities include:

- Amanda (misc/amanda-server and misc/amanda-client),
- Bacula (sysutils/bacula13-server and sysutils/bacula13-client),
- Bareos (sysutils/bareos-server and sysutils/bareos-client),
- net/rsync,
- sysutils/duply, and
- sysutils/duplicity.

20.8.5. Emergency Recovery

In addition to regular backups, it is recommended to perform the following steps as part of an emergency preparedness plan.

Create a print copy of the output of the following commands:

- `gpart show`
- `more /etc/fstab`
- `pkg prime-list`
- `dmesg`

Store this printout and a copy of the installation media in a secure location. Should an emergency restore be needed, boot into the installation media and select `Live CD` to access a rescue shell. This rescue mode can be used to view the current state of the system, and if needed, to reformat disks and restore data from backups.

Next, test the rescue shell and the backups. Make notes of the procedure. Store these notes with the media, the printouts, and the backups. These notes may prevent the inadvertent destruction of the backups while under the stress of performing an emergency recovery.

For an added measure of security, store the latest backup at a remote location which is physically separated from the computers and disk drives by a significant distance.

20.9. Memory Disks

In addition to physical disks, FreeBSD also supports the creation and use of memory disks. One possible use for a memory disk is to access the contents of an ISO file system without the overhead of first burning it to a CD or DVD, then mounting the CD/DVD media.

In FreeBSD, the md(4) driver is used to provide support for memory disks. The GENERIC kernel includes this driver. When using a custom kernel configuration file, ensure it includes this line:

```
device md
```

20.9.1. Attaching and Detaching Existing Images

To mount an existing file system image, use `mdconfig` to specify the name of the ISO file and a free unit number. Then, refer to that unit number to mount it on an existing mount point. Once mounted, the files in the ISO will appear in the mount point. This example attaches *diskimage.iso* to the memory device /dev/md0 then mounts that memory device on /mnt:

```
# mdconfig -f diskimage.iso -u 0
# mount -t cd9660 /dev/md0 /mnt
```

Notice that `-t cd9660` was used to mount an ISO format. If a unit number is not specified with `-u`, `mdconfig` will automatically allocate an unused memory device and output the name of the allocated unit, such as md4. Refer to mdconfig(8) for more details about this command and its options.

When a memory disk is no longer in use, its resources should be released back to the system. First, unmount the file system, then use `mdconfig` to detach the disk from the system and release its resources. To continue this example:

```
# umount /mnt
# mdconfig -d -u 0
```

To determine if any memory disks are still attached to the system, type `mdconfig -l`.

20.9.2. Creating a File- or Memory-Backed Memory Disk

FreeBSD also supports memory disks where the storage to use is allocated from either a hard disk or an area of memory. The first method is commonly referred to as a file-backed file system and the second method as a memory-backed file system. Both types can be created using `mdconfig`.

To create a new memory-backed file system, specify a type of `swap` and the size of the memory disk to create. Then, format the memory disk with a file system and mount as usual. This example creates a 5M memory disk on unit `1`. That memory disk is then formatted with the UFS file system before it is mounted:

```
# mdconfig -a -t swap -s 5m -u 1
# newfs -U md1
/dev/md1: 5.0MB (10240 sectors) block size 16384, fragment size 2048
        using 4 cylinder groups of 1.27MB, 81 blks, 192 inodes.
        with soft updates
super-block backups (for fsck -b #) at:
 160, 2752, 5344, 7936
```

```
# mount /dev/md1 /mnt
# df /mnt
Filesystem 1K-blocks Used Avail Capacity  Mounted on
/dev/md1        4718    4  4338     0%    /mnt
```

To create a new file-backed memory disk, first allocate an area of disk to use. This example creates an empty 5MB file named newimage:

```
# dd if=/dev/zero of=newimage bs=1k count=5k
5120+0 records in
5120+0 records out
```

Next, attach that file to a memory disk, label the memory disk and format it with the UFS file system, mount the memory disk, and verify the size of the file-backed disk:

```
# mdconfig -f newimage -u 0
# bsdlabel -w md0 auto
# newfs -U md0a
/dev/md0a: 5.0MB (10224 sectors) block size 16384, fragment size 2048
        using 4 cylinder groups of 1.25MB, 80 blks, 192 inodes.
super-block backups (for fsck -b #) at:
 160, 2720, 5280, 7840
# mount /dev/md0a /mnt
# df /mnt
Filesystem 1K-blocks Used Avail Capacity  Mounted on
/dev/md0a       4710    4  4330     0%    /mnt
```

It takes several commands to create a file- or memory-backed file system using `mdconfig`. FreeBSD also comes with `mdmfs` which automatically configures a memory disk, formats it with the UFS file system, and mounts it. For example, after creating *newimage* with `dd`, this one command is equivalent to running the `bsdlabel`, `newfs`, and `mount` commands shown above:

```
# mdmfs -F newimage -s 5m md0 /mnt
```

To instead create a new memory-based memory disk with `mdmfs`, use this one command:

```
# mdmfs -s 5m md1 /mnt
```

If the unit number is not specified, `mdmfs` will automatically select an unused memory device. For more details about `mdmfs`, refer to mdmfs(8).

20.10. File System Snapshots

FreeBSD offers a feature in conjunction with soft updates: file system snapshots.

UFS snapshots allow a user to create images of specified file systems, and treat them as a file. When using the Z file system (ZFS), refer to Managing Snapshots on how to use snapshots.

Snapshot files must be created in the file system that the action is performed on, and a user may create no more than 20 snapshots per file system. Active snapshots are recorded in the superblock so they are persistent across unmount and remount operations along with system reboots. When a snapshot is no longer required, it can be removed using rm(1). While snapshots may be removed in any order, all the used space may not be acquired because another snapshot will possibly claim some of the released blocks.

The un-alterable `snapshot` file flag is set by mksnap_ffs(8) after initial creation of a snapshot file. unlink(1) makes an exception for snapshot files since it allows them to be removed.

Snapshots are created using mount(8). To place a snapshot of /var in the file /var/snapshot/snap, use the following command:

```
# mount -u -o snapshot /var/snapshot/snap /var
```

Alternatively, use mksnap_ffs(8) to create the snapshot:

```
# mksnap_ffs /var /var/snapshot/snap
```

One can find snapshot files on a file system, such as /var, using find(1):

```
# find /var -flags snapshot
```

Once a snapshot has been created, it has several uses:

- Some administrators will use a snapshot file for backup purposes, because the snapshot can be transferred to CDs or tape.
- The file system integrity checker, fsck(8), may be run on the snapshot. Assuming that the file system was clean when it was mounted, this should always provide a clean and unchanging result.
- Running dump(8) on the snapshot will produce a dump file that is consistent with the file system and the timestamp of the snapshot. dump(8) can also take a snapshot, create a dump image, and then remove the snapshot in one command by using `-L`.
- The snapshot can be mounted as a frozen image of the file system. To mount(8) the snapshot /var/snapshot/snap run:

  ```
  # mdconfig -a -t vnode -o readonly -f /var/snapshot/snap -u 4
  # mount -r /dev/md4 /mnt
  ```

The frozen /var is now available through /mnt. Everything will initially be in the same state it was during the snapshot creation time. The only exception is that any earlier snapshots will appear as

zero length files. To unmount the snapshot, use:

```
# umount /mnt
# mdconfig -d -u 4
```

For more information about `softupdates` and file system snapshots, including technical papers, visit Marshall Kirk McKusick's website at http://www.mckusick.com/.

20.11. Disk Quotas

Disk quotas can be used to limit the amount of disk space or the number of files a user or members of a group may allocate on a per-file system basis. This prevents one user or group of users from consuming all of the available disk space.

This section describes how to configure disk quotas for the UFS file system. To configure quotas on the ZFS file system, refer to Dataset, User, and Group Quotas

20.11.1. Enabling Disk Quotas

To determine if the FreeBSD kernel provides support for disk quotas:

```
% sysctl kern.features.ufs_quota
kern.features.ufs_quota: 1
```

In this example, the `1` indicates quota support. If the value is instead `0`, add the following line to a custom kernel configuration file and rebuild the kernel using the instructions in Configuring the FreeBSD Kernel:

```
options QUOTA
```

Next, enable disk quotas in /etc/rc.conf:

```
quota_enable="YES"
```

Normally on bootup, the quota integrity of each file system is checked by quotacheck(8). This program insures that the data in the quota database properly reflects the data on the file system. This is a time consuming process that will significantly affect the time the system takes to boot. To skip this step, add this variable to /etc/rc.conf:

```
check_quotas="NO"
```

Finally, edit /etc/fstab to enable disk quotas on a per-file system basis. To enable per-user quotas on a file system, add `userquota` to the options field in the /etc/fstab entry for the file system to enable quotas on. For example:

```
/dev/da1s2g   /home    ufs rw,userquota 1 2
```

To enable group quotas, use `groupquota` instead. To enable both user and group quotas, separate the options with a comma:

```
/dev/da1s2g    /home    ufs rw,userquota,groupquota 1 2
```

By default, quota files are stored in the root directory of the file system as quota.user and quota.group. Refer to fstab(5) for more information. Specifying an alternate location for the quota files is not recommended.

Once the configuration is complete, reboot the system and /etc/rc will automatically run the appropriate commands to create the initial quota files for all of the quotas enabled in /etc/fstab.

In the normal course of operations, there should be no need to manually run quotacheck(8), quotaon(8), or quotaoff(8). However, one should read these manual pages to be familiar with their operation.

20.11.2. Setting Quota Limits

To verify that quotas are enabled, run:

```
# quota -v
```

There should be a one line summary of disk usage and current quota limits for each file system that quotas are enabled on.

The system is now ready to be assigned quota limits with `edquota`.

Several options are available to enforce limits on the amount of disk space a user or group may allocate, and how many files they may create. Allocations can be limited based on disk space (block quotas), number of files (inode quotas), or a combination of both. Each limit is further broken down into two categories: hard and soft limits.

A hard limit may not be exceeded. Once a user reaches a hard limit, no further allocations can be made on that file system by that user. For example, if the user has a hard limit of 500 kbytes on a file system and is currently using 490 kbytes, the user can only allocate an additional 10 kbytes. Attempting to allocate an additional 11 kbytes will fail.

Soft limits can be exceeded for a limited amount of time, known as the grace period, which is one week by default. If a user stays over their limit longer than the grace period, the soft limit turns into a hard limit and no further allocations are allowed. When the user drops back below the soft limit, the grace period is reset.

In the following example, the quota for the `test` account is being edited. When `edquota` is invoked, the editor specified by `EDITOR` is opened in order to edit the quota limits. The default editor is set to vi.

```
# edquota -u test
Quotas for user test:
/usr: kbytes in use: 65, limits (soft = 50, hard = 75)
        inodes in use: 7, limits (soft = 50, hard = 60)
/usr/var: kbytes in use: 0, limits (soft = 50, hard = 75)
        inodes in use: 0, limits (soft = 50, hard = 60)
```

There are normally two lines for each file system that has quotas enabled. One line represents the block limits and the other represents the inode limits. Change the value to modify the quota limit. For example, to raise the block limit on /usr to a soft limit of `500` and a hard limit of `600`, change the values in that line as follows:

```
/usr: kbytes in use: 65, limits (soft = 500, hard = 600)
```

The new quota limits take effect upon exiting the editor.

Sometimes it is desirable to set quota limits on a range of users. This can be done by first assigning the desired quota limit to a user. Then, use `-p` to duplicate that quota to a specified range of user IDs (UIDs). The following command will duplicate those quota limits for UIDs `10,000` through `19,999`:

```
# edquota -p test 10000-19999
```

For more information, refer to edquota(8).

20.11.3. Checking Quota Limits and Disk Usage

To check individual user or group quotas and disk usage, use quota(1). A user may only examine their own quota and the quota of a group they are a member of. Only the superuser may view all user and group quotas. To get a summary of all quotas and disk usage for file systems with quotas enabled, use repquota(8).

Normally, file systems that the user is not using any disk space on will not show in the output of `quota`, even if the user has a quota limit assigned for that file system. Use `-v` to display those file systems. The following is sample output from `quota -v` for a user that has quota limits on two file systems.

```
Disk quotas for user test (uid 1002):
     Filesystem  usage    quota   limit   grace   files   quota   limit   grace
           /usr      65*     50      75   5days       7      50      60
       /usr/var       0      50      75               0      50      60
```

In this example, the user is currently 15 kbytes over the soft limit of 50 kbytes on /usr and has 5 days of grace period left. The asterisk * indicates that the user is currently over the quota limit.

20.11.4. Quotas over NFS

Quotas are enforced by the quota subsystem on the NFS server. The rpc.rquotad(8) daemon makes quota information available to `quota` on NFS clients, allowing users on those machines to see their quota statistics.

On the NFS server, enable `rpc.rquotad` by removing the `#` from this line in **/etc/inetd.conf**:

```
rquotad/1      dgram rpc/udp wait root /usr/libexec/rpc.rquotad rpc.rquotad
```

Then, restart `inetd`:

```
# service inetd restart
```

20.12. Encrypting Disk Partitions

FreeBSD offers excellent online protections against unauthorized data access. File permissions and Mandatory Access Control (MAC) help prevent unauthorized users from accessing data while the operating system is active and the computer is powered up. However, the permissions enforced by the operating system are irrelevant if an attacker has physical access to a computer and can move the computer's hard drive to another system to copy and analyze the data.

Regardless of how an attacker may have come into possession of a hard drive or powered-down computer, the GEOM-based cryptographic subsystems built into FreeBSD are able to protect the data on the computer's file systems against even highly-motivated attackers with significant resources. Unlike encryption methods that encrypt individual files, the built-in `gbde` and `geli` utilities can be used to transparently encrypt entire file systems. No cleartext ever touches the hard drive's platter.

This chapter demonstrates how to create an encrypted file system on FreeBSD. It first demonstrates the process using `gbde` and then demonstrates the same example using `geli`.



GBDE disk-encryption is deprecated, and removed in FreeBSD 15.0.

20.12.1. Disk Encryption with gbde

The objective of the gbde(4) facility is to provide a formidable challenge for an attacker to gain access to the contents of a *cold* storage device. However, if the computer is compromised while up and running and the storage device is actively attached, or the attacker has access to a valid passphrase, it offers no protection to the contents of the storage device. Thus, it is important to provide physical security while the system is running and to protect the passphrase used by the encryption mechanism.

This facility provides several barriers to protect the data stored in each disk sector. It encrypts the contents of a disk sector using 128-bit AES in CBC mode. Each sector on the disk is encrypted with a different AES key. For more information on the cryptographic design, including how the sector keys

are derived from the user-supplied passphrase, refer to gbde(4).

FreeBSD provides a kernel module for gbde which can be loaded with this command:

```
# kldload geom_bde
```

If using a custom kernel configuration file, ensure it contains this line:

```
options GEOM_BDE
```

The following example demonstrates adding a new hard drive to a system that will hold a single encrypted partition that will be mounted as /private.

Procedure: Encrypting a Partition with gbde

1. Add the New Hard Drive

   Install the new drive to the system as explained in Adding Disks. For the purposes of this example, a new hard drive partition has been added as /dev/ad4s1c and /dev/ad0s1* represents the existing standard FreeBSD partitions.

   ```
   # ls /dev/ad*
   /dev/ad0        /dev/ad0s1b     /dev/ad0s1e     /dev/ad4s1
   /dev/ad0s1      /dev/ad0s1c     /dev/ad0s1f     /dev/ad4s1c
   /dev/ad0s1a     /dev/ad0s1d     /dev/ad4
   ```

2. Create a Directory to Hold `gbde` Lock Files

   ```
   # mkdir /etc/gbde
   ```

   The gbde lock file contains information that gbde requires to access encrypted partitions. Without access to the lock file, gbde will not be able to decrypt the data contained in the encrypted partition without significant manual intervention which is not supported by the software. Each encrypted partition uses a separate lock file.

3. Initialize the `gbde` Partition

   A gbde partition must be initialized before it can be used. This initialization needs to be performed only once. This command will open the default editor, in order to set various configuration options in a template. For use with the UFS file system, set the sector_size to 2048:

   ```
   # gbde init /dev/ad4s1c -i -L /etc/gbde/ad4s1c.lock
   #
   # Sector size is the smallest unit of data which can be read or written.
   # Making it too small decreases performance and decreases available space.
   # Making it too large may prevent filesystems from working.  512 is the
   # minimum and always safe.  For UFS, use the fragment size
   #
   ```

```
sector_size =   2048
[...]
```

Once the edit is saved, the user will be asked twice to type the passphrase used to secure the data. The passphrase must be the same both times. The ability of gbde to protect data depends entirely on the quality of the passphrase. For tips on how to select a secure passphrase that is easy to remember, see http://world.std.com/~reinhold/diceware.htm.

This initialization creates a lock file for the gbde partition. In this example, it is stored as /etc/gbde/ad4s1c.lock. Lock files must end in ".lock" in order to be correctly detected by the /etc/rc.d/gbde start up script.



Lock files *must* be backed up together with the contents of any encrypted partitions. Without the lock file, the legitimate owner will be unable to access the data on the encrypted partition.

4. Attach the Encrypted Partition to the Kernel

```
# gbde attach /dev/ad4s1c -l /etc/gbde/ad4s1c.lock
```

This command will prompt to input the passphrase that was selected during the initialization of the encrypted partition. The new encrypted device will appear in /dev as /dev/device_name.bde:

```
# ls /dev/ad*
/dev/ad0        /dev/ad0s1b     /dev/ad0s1e     /dev/ad4s1
/dev/ad0s1      /dev/ad0s1c     /dev/ad0s1f     /dev/ad4s1c
/dev/ad0s1a     /dev/ad0s1d     /dev/ad4        /dev/ad4s1c.bde
```

5. Create a File System on the Encrypted Device

Once the encrypted device has been attached to the kernel, a file system can be created on the device. This example creates a UFS file system with soft updates enabled. Be sure to specify the partition which has a *.bde extension:

```
# newfs -U /dev/ad4s1c.bde
```

6. Mount the Encrypted Partition

Create a mount point and mount the encrypted file system:

```
# mkdir /private
# mount /dev/ad4s1c.bde /private
```

7. Verify That the Encrypted File System is Available

The encrypted file system should now be visible and available for use:

```
% df -H
Filesystem         Size   Used  Avail Capacity  Mounted on
/dev/ad0s1a       1037M    72M   883M     8%    /
/devfs             1.0K   1.0K     0B   100%    /dev
/dev/ad0s1f        8.1G    55K   7.5G     0%    /home
/dev/ad0s1e       1037M   1.1M   953M     0%    /tmp
/dev/ad0s1d        6.1G   1.9G   3.7G    35%    /usr
/dev/ad4s1c.bde    150G   4.1K   138G     0%    /private
```

After each boot, any encrypted file systems must be manually re-attached to the kernel, checked for errors, and mounted, before the file systems can be used. To configure these steps, add the following lines to /etc/rc.conf:

```
gbde_autoattach_all="YES"
gbde_devices="ad4s1c"
gbde_lockdir="/etc/gbde"
```

This requires that the passphrase be entered at the console at boot time. After typing the correct passphrase, the encrypted partition will be mounted automatically. Additional gbde boot options are available and listed in rc.conf(5).



sysinstall is incompatible with gbde-encrypted devices. All *.bde devices must be detached from the kernel before starting sysinstall or it will crash during its initial probing for devices. To detach the encrypted device used in the example, use the following command:

```
# gbde detach /dev/ad4s1c
```

20.12.2. Disk Encryption with `geli`

An alternative cryptographic GEOM class is available using `geli`. This control utility adds some features and uses a different scheme for doing cryptographic work. It provides the following features:

- Utilizes the crypto(9) framework and automatically uses cryptographic hardware when it is available.
- Supports multiple cryptographic algorithms such as AES-XTS, AES-CBC, and Camellia-CBCAES.
- Allows the root partition to be encrypted. The passphrase used to access the encrypted root partition will be requested during system boot.
- Allows the use of two independent keys.
- It is fast as it performs simple sector-to-sector encryption.
- Allows backup and restore of master keys. If a user destroys their keys, it is still possible to get

access to the data by restoring keys from the backup.

- Allows a disk to attach with a random, one-time key which is useful for swap partitions and temporary file systems.

More features and usage examples can be found in geli(8).

The following example describes how to generate a key file which will be used as part of the master key for the encrypted provider mounted under /private. The key file will provide some random data used to encrypt the master key. The master key will also be protected by a passphrase. The provider's sector size will be 4kB. The example describes how to attach to the `geli` provider, create a file system on it, mount it, work with it, and finally, how to detach it.

Procedure: Encrypting a Partition with `geli`

1. Load `geli` Support

   Support for `geli` is available as a loadable kernel module. To configure the system to automatically load the module at boot time, add the following line to /boot/loader.conf:

   ```
   geom_eli_load="YES"
   ```

   To load the kernel module now:

   ```
   # kldload geom_eli
   ```

   For a custom kernel, ensure the kernel configuration file contains these lines:

   ```
   options GEOM_ELI
   device crypto
   ```

2. Generate the Master Key

   The following commands generate a master key that all data will be encrypted with. This key can never be changed. Rather than using it directly, it is encrypted with one or more user keys. The user keys are made up of an optional combination of random bytes from a file, /root/da2.key, and/or a passphrase. In this case, the data source for the key file is /dev/random. This command also configures the sector size of the provider (/dev/da2.eli) as 4kB, for better performance:

   ```
   # dd if=/dev/random of=/root/da2.key bs=64 count=1
   # geli init -K /root/da2.key -s 4096 /dev/da2
   Enter new passphrase:
   Reenter new passphrase:
   ```

   It is not mandatory to use both a passphrase and a key file as either method of securing the master key can be used in isolation.

If the key file is given as "-", standard input will be used. For example, this command generates three key files:

```
# cat keyfile1 keyfile2 keyfile3 | geli init -K - /dev/da2
```

3. Attach the Provider with the Generated Key

   To attach the provider, specify the key file, the name of the disk, and the passphrase:

```
# geli attach -k /root/da2.key /dev/da2
Enter passphrase:
```

   This creates a new device with an .eli extension:

```
# ls /dev/da2*
/dev/da2  /dev/da2.eli
```

4. Create the New File System

   Next, format the device with the UFS file system and mount it on an existing mount point:

```
# dd if=/dev/random of=/dev/da2.eli bs=1m
# newfs /dev/da2.eli
# mount /dev/da2.eli /private
```

   The encrypted file system should now be available for use:

```
# df -H
Filesystem     Size   Used  Avail Capacity  Mounted on
/dev/ad0s1a    248M    89M   139M    38%    /
/devfs         1.0K   1.0K     0B   100%    /dev
/dev/ad0s1f    7.7G   2.3G   4.9G    32%    /usr
/dev/ad0s1d    989M   1.5M   909M     0%    /tmp
/dev/ad0s1e    3.9G   1.3G   2.3G    35%    /var
/dev/da2.eli   150G   4.1K   138G     0%    /private
```

Once the work on the encrypted partition is done, and the /private partition is no longer needed, it is prudent to put the device into cold storage by unmounting and detaching the `geli` encrypted partition from the kernel:

```
# umount /private
# geli detach da2.eli
```

An rc.d script is provided to simplify the mounting of `geli`-encrypted devices at boot time. For this

example, add these lines to /etc/rc.conf:

```
geli_devices="da2"
geli_da2_flags="-k /root/da2.key"
```

This configures /dev/da2 as a `geli` provider with a master key of /root/da2.key. The system will automatically detach the provider from the kernel before the system shuts down. During the startup process, the script will prompt for the passphrase before attaching the provider. Other kernel messages might be shown before and after the password prompt. If the boot process seems to stall, look carefully for the password prompt among the other messages. Once the correct passphrase is entered, the provider is attached. The file system is then mounted, typically by an entry in /etc/fstab. Refer to "Mounting and Unmounting File Systems" for instructions on how to configure a file system to mount at boot time.

20.13. Encrypting Swap

Like the encryption of disk partitions, encryption of swap space is used to protect sensitive information. Consider an application that deals with passwords. As long as these passwords stay in physical memory, they are not written to disk and will be cleared after a reboot. However, if FreeBSD starts swapping out memory pages to free space, the passwords may be written to the disk unencrypted. Encrypting swap space can be a solution for this scenario.

This section demonstrates how to configure an encrypted swap partition using gbde(8) or geli(8) encryption. It assumes that /dev/ada0s1b is the swap partition.

20.13.1. Configuring Encrypted Swap

Swap partitions are not encrypted by default and should be cleared of any sensitive data before continuing. To overwrite the current swap partition with random garbage, execute the following command:

```
# dd if=/dev/random of=/dev/ada0s1b bs=1m
```

To encrypt the swap partition using gbde(8), add the `.bde` suffix to the swap line in /etc/fstab:

```
# Device        Mountpoint  FStype  Options     Dump    Pass#
/dev/ada0s1b.bde    none        swap    sw      0   0
```

To instead encrypt the swap partition using geli(8), use the `.eli` suffix:

```
# Device        Mountpoint  FStype  Options     Dump    Pass#
/dev/ada0s1b.eli    none        swap    sw      0   0
```

By default, geli(8) uses the AES algorithm with a key length of 128 bits. Normally the default settings

will suffice. If desired, these defaults can be altered in the options field in /etc/fstab. The possible flags are:

aalgo

Data integrity verification algorithm used to ensure that the encrypted data has not been tampered with. See geli(8) for a list of supported algorithms.

ealgo

Encryption algorithm used to protect the data. See geli(8) for a list of supported algorithms.

keylen

The length of the key used for the encryption algorithm. See geli(8) for the key lengths that are supported by each encryption algorithm.

sectorsize

The size of the blocks data is broken into before it is encrypted. Larger sector sizes increase performance at the cost of higher storage overhead. The recommended size is 4096 bytes.

This example configures an encrypted swap partition using the AES-XTS algorithm with a key length of 128 bits and a sectorsize of 4 kilobytes:

```
# Device        Mountpoint  FStype  Options             Dump    Pass#
/dev/ada0s1b.eli    none        swap    sw,ealgo=AES-XTS,keylen=128,sectorsize=4096 0
0
```

20.13.2. Encrypted Swap Verification

Once the system has rebooted, proper operation of the encrypted swap can be verified using `swapinfo`.

If gbde(8) is being used:

```
% swapinfo
Device          1K-blocks     Used    Avail Capacity
/dev/ada0s1b.bde   542720        0   542720     0
```

If geli(8) is being used:

```
% swapinfo
Device          1K-blocks     Used    Avail Capacity
/dev/ada0s1b.eli   542720        0   542720     0
```

20.14. Highly Available Storage (HAST)

High availability is one of the main requirements in serious business applications and highly-

available storage is a key component in such environments. In FreeBSD, the Highly Available STorage (HAST) framework allows transparent storage of the same data across several physically separated machines connected by a TCP/IP network. HAST can be understood as a network-based RAID1 (mirror), and is similar to the DRBD® storage system used in the GNU/Linux® platform. In combination with other high-availability features of FreeBSD like CARP, HAST makes it possible to build a highly-available storage cluster that is resistant to hardware failures.

The following are the main features of HAST:

- Can be used to mask I/O errors on local hard drives.
- File system agnostic as it works with any file system supported by FreeBSD.
- Efficient and quick resynchronization as only the blocks that were modified during the downtime of a node are synchronized.
- Can be used in an already deployed environment to add additional redundancy.
- Together with CARP, Heartbeat, or other tools, it can be used to build a robust and durable storage system.

Read this chapter to learn:

- What HAST is, how it works, and which features it provides.
- How to set up and use HAST on FreeBSD.
- How to integrate CARP and devd(8) to build a robust storage system.

Before reading this section:

- Understand UNIX® and FreeBSD basics (FreeBSD Basics).
- Know how to configure network interfaces and other core FreeBSD subsystems (Configuration and Tuning).
- Have a good understanding of FreeBSD networking (Network Communication).

The HAST project was sponsored by The FreeBSD Foundation with support from http://www.omc.net/ and http://www.transip.nl/.

20.14.1. HAST Operation

HAST provides synchronous block-level replication between two physical machines: the *primary* node and the *secondary* node. These two machines together are referred to as a cluster.

Since HAST works in a primary-secondary configuration, it allows only one of the cluster nodes to be active at any given time. The primary node, also called *active*, is the one which will handle all the I/O requests to HAST-managed devices. The secondary node is automatically synchronized from the primary node.

The physical components of the HAST system are the local disk on primary node, and the disk on the remote, secondary node.

HAST operates synchronously on a block level, making it transparent to file systems and

applications. HAST provides regular GEOM providers in /dev/hast/ for use by other tools or applications. There is no difference between using HAST-provided devices and raw disks or partitions.

Each write, delete, or flush operation is sent to both the local disk and to the remote disk over TCP/IP. Each read operation is served from the local disk, unless the local disk is not up-to-date or an I/O error occurs. In such cases, the read operation is sent to the secondary node.

HAST tries to provide fast failure recovery. For this reason, it is important to reduce synchronization time after a node's outage. To provide fast synchronization, HAST manages an on-disk bitmap of dirty extents and only synchronizes those during a regular synchronization, with an exception of the initial sync.

There are many ways to handle synchronization. HAST implements several replication modes to handle different synchronization methods:

- *memsync*: This mode reports a write operation as completed when the local write operation is finished and when the remote node acknowledges data arrival, but before actually storing the data. The data on the remote node will be stored directly after sending the acknowledgement. This mode is intended to reduce latency, but still provides good reliability. This mode is the default.
- *fullsync*: This mode reports a write operation as completed when both the local write and the remote write complete. This is the safest and the slowest replication mode.
- *async*: This mode reports a write operation as completed when the local write completes. This is the fastest and the most dangerous replication mode. It should only be used when replicating to a distant node where latency is too high for other modes.

20.14.2. HAST Configuration

The HAST framework consists of several components:

- The hastd(8) daemon which provides data synchronization. When this daemon is started, it will automatically load `geom_gate.ko`.
- The userland management utility, hastctl(8).
- The hast.conf(5) configuration file. This file must exist before starting hastd.

Users who prefer to statically build `GEOM_GATE` support into the kernel should add this line to the custom kernel configuration file, then rebuild the kernel using the instructions in Configuring the FreeBSD Kernel:

```
options GEOM_GATE
```

The following example describes how to configure two nodes in primary-secondary operation using HAST to replicate the data between the two. The nodes will be called `hasta`, with an IP address of `172.16.0.1`, and `hastb`, with an IP address of `172.16.0.2`. Both nodes will have a dedicated hard drive /dev/ad6 of the same size for HAST operation. The HAST pool, sometimes referred to as a resource or the GEOM provider in /dev/hast/, will be called `test`.

Configuration of HAST is done using /etc/hast.conf. This file should be identical on both nodes. The simplest configuration is:

```
resource test {
    on hasta {
        local /dev/ad6
        remote 172.16.0.2
    }
    on hastb {
        local /dev/ad6
        remote 172.16.0.1
    }
}
```

For more advanced configuration, refer to hast.conf(5).



It is also possible to use host names in the `remote` statements if the hosts are resolvable and defined either in /etc/hosts or in the local DNS.

Once the configuration exists on both nodes, the HAST pool can be created. Run these commands on both nodes to place the initial metadata onto the local disk and to start hastd(8):

```
# hastctl create test
# service hastd onestart
```



It is *not* possible to use GEOM providers with an existing file system or to convert an existing storage to a HAST-managed pool. This procedure needs to store some metadata on the provider and there will not be enough required space available on an existing provider.

A HAST node's `primary` or `secondary` role is selected by an administrator, or software like Heartbeat, using hastctl(8). On the primary node, `hasta`, issue this command:

```
# hastctl role primary test
```

Run this command on the secondary node, `hastb`:

```
# hastctl role secondary test
```

Verify the result by running `hastctl` on each node:

```
# hastctl status test
```

Check the `status` line in the output. If it says `degraded`, something is wrong with the configuration

file. It should say `complete` on each node, meaning that the synchronization between the nodes has started. The synchronization completes when `hastctl status` reports 0 bytes of `dirty` extents.

The next step is to create a file system on the GEOM provider and mount it. This must be done on the `primary` node. Creating the file system can take a few minutes, depending on the size of the hard drive. This example creates a UFS file system on /dev/hast/test:

```
# newfs -U /dev/hast/test
# mkdir /hast/test
# mount /dev/hast/test /hast/test
```

Once the HAST framework is configured properly, the final step is to make sure that HAST is started automatically during system boot. Add this line to /etc/rc.conf:

```
hastd_enable="YES"
```

20.14.2.1. Failover Configuration

The goal of this example is to build a robust storage system which is resistant to the failure of any given node. If the primary node fails, the secondary node is there to take over seamlessly, check and mount the file system, and continue to work without missing a single bit of data.

To accomplish this task, the Common Address Redundancy Protocol (CARP) is used to provide for automatic failover at the IP layer. CARP allows multiple hosts on the same network segment to share an IP address. Set up CARP on both nodes of the cluster according to the documentation available in "Common Address Redundancy Protocol (CARP)". In this example, each node will have its own management IP address and a shared IP address of *172.16.0.254*. The primary HAST node of the cluster must be the primary CARP node.

The HAST pool created in the previous section is now ready to be exported to the other hosts on the network. This can be accomplished by exporting it through NFS or Samba, using the shared IP address *172.16.0.254*. The only problem which remains unresolved is an automatic failover should the primary node fail.

In the event of CARP interfaces going up or down, the FreeBSD operating system generates a devd(8) event, making it possible to watch for state changes on the CARP interfaces. A state change on the CARP interface is an indication that one of the nodes failed or came back online. These state change events make it possible to run a script which will automatically handle the HAST failover.

To catch state changes on the CARP interfaces, add this configuration to /etc/devd.conf on each node, while replacing `<vhid>` with the virtual host id and `<ifname>` with the associated interface name:

```
notify 30 {
    match "system" "CARP";
    match "subsystem" "<vhid>@<ifname>";
    match "type" "MASTER";
```

```
    action "/usr/local/sbin/carp-hast-switch primary";
};

notify 30 {
    match "system" "CARP";
    match "subsystem" "<vhid>@<ifname>";
    match "type" "BACKUP";
    action "/usr/local/sbin/carp-hast-switch secondary";
};
```

Restart devd(8) on both nodes to put the new configuration into effect:

```
# service devd restart
```

When the specified interface state changes by going up or down , the system generates a notification, allowing the devd(8) subsystem to run the specified automatic failover script, /usr/local/sbin/carp-hast-switch. For further clarification about this configuration, refer to devd.conf(5).

Here is an example of an automated failover script:

```
#!/bin/sh

# Original script by Freddie Cash <fjwcash@gmail.com>
# Modified by Michael W. Lucas <mwlucas@BlackHelicopters.org>
# and Viktor Petersson <vpetersson@wireload.net>

# The names of the HAST resources, as listed in /etc/hast.conf
resources="test"

# delay in mounting HAST resource after becoming primary
# make your best guess
delay=3

# logging
log="local0.debug"
name="carp-hast"

# end of user configurable stuff

case "$1" in
    primary)
        logger -p $log -t $name "Switching to primary provider for ${resources}."
        sleep ${delay}

        # Wait for any "hastd secondary" processes to stop
        for disk in ${resources}; do
            while $( pgrep -lf "hastd: ${disk} \(secondary\)" > /dev/null 2>&1 ); do
                sleep 1
```

```
            done

            # Switch role for each disk
            hastctl role primary ${disk}
            if [ $? -ne 0 ]; then
                logger -p $log -t $name "Unable to change role to primary for resource
${disk}."
                exit 1
            fi
        done

        # Wait for the /dev/hast/* devices to appear
        for disk in ${resources}; do
            for I in $( jot 60 ); do
                [ -c "/dev/hast/${disk}" ] && break
                sleep 0.5
            done

            if [ ! -c "/dev/hast/${disk}" ]; then
                logger -p $log -t $name "GEOM provider /dev/hast/${disk} did not
appear."
                exit 1
            fi
        done

        logger -p $log -t $name "Role for HAST resources ${resources} switched to
primary."

        logger -p $log -t $name "Mounting disks."
        for disk in ${resources}; do
            mkdir -p /hast/${disk}
            fsck -p -y -t ufs /dev/hast/${disk}
            mount /dev/hast/${disk} /hast/${disk}
        done

    ;;

    secondary)
        logger -p $log -t $name "Switching to secondary provider for ${resources}."

        # Switch roles for the HAST resources
        for disk in ${resources}; do
            if ! mount | grep -q "^/dev/hast/${disk} on "
            then
            else
                umount -f /hast/${disk}
            fi
            sleep $delay
            hastctl role secondary ${disk} 2>&1
            if [ $? -ne 0 ]; then
                logger -p $log -t $name "Unable to switch role to secondary for
```

```
resource ${disk}."
                exit 1
            fi
            logger -p $log -t $name "Role switched to secondary for resource ${disk}."
        done
    ;;
esac
```

In a nutshell, the script takes these actions when a node becomes primary:

- Promotes the HAST pool to primary on the other node.
- Checks the file system under the HAST pool.
- Mounts the pool.

When a node becomes secondary:

- Unmounts the HAST pool.
- Degrades the HAST pool to secondary.



This is just an example script which serves as a proof of concept. It does not handle all the possible scenarios and can be extended or altered in any way, for example, to start or stop required services.



For this example, a standard UFS file system was used. To reduce the time needed for recovery, a journal-enabled UFS or ZFS file system can be used instead.

Instead of using the highly available storage locally, it can also be shared to other computers on a network via NFS, iSCSI, sshfs(1), or programs in ports (i.e. net/samba419).

More detailed information with additional examples can be found at http://wiki.FreeBSD.org/HAST.

20.14.3. Troubleshooting

HAST should generally work without issues. However, as with any other software product, there may be times when it does not work as supposed. The sources of the problems may be different, but the rule of thumb is to ensure that the time is synchronized between the nodes of the cluster.

When troubleshooting HAST, the debugging level of hastd(8) should be increased by starting `hastd` with `-d`. This argument may be specified multiple times to further increase the debugging level. Consider also using `-F`, which starts `hastd` in the foreground.

20.14.3.1. Recovering from the Split-brain Condition

Split-brain occurs when the nodes of the cluster are unable to communicate with each other, and both are configured as primary. This is a dangerous condition because it allows both nodes to make incompatible changes to the data. This problem must be corrected manually by the system administrator.

The administrator must either decide which node has more important changes, or perform the merge manually. Then, let HAST perform full synchronization of the node which has the broken data. To do this, issue these commands on the node which needs to be resynchronized:

```
# hastctl role init test
# hastctl create test
# hastctl role secondary test
```

Chapter 21. GEOM: Modular Disk Transformation Framework

21.1. Synopsis

In FreeBSD, the GEOM framework permits access and control to classes, such as Master Boot Records and BSD labels, through the use of providers, or the disk devices in /dev. By supporting various software RAID configurations, GEOM transparently provides access to the operating system and operating system utilities.

This chapter covers the use of disks under the GEOM framework in FreeBSD. This includes the major RAID control utilities which use the framework for configuration. This chapter is not a definitive guide to RAID configurations and only GEOM-supported RAID classifications are discussed.

Read this chapter to learn:

- What type of RAID support is available through GEOM.
- How to use the base utilities to configure, maintain, and manipulate the various RAID levels.
- How to mirror, stripe, encrypt, and remotely connect disk devices through GEOM.
- How to troubleshoot disks attached to the GEOM framework.

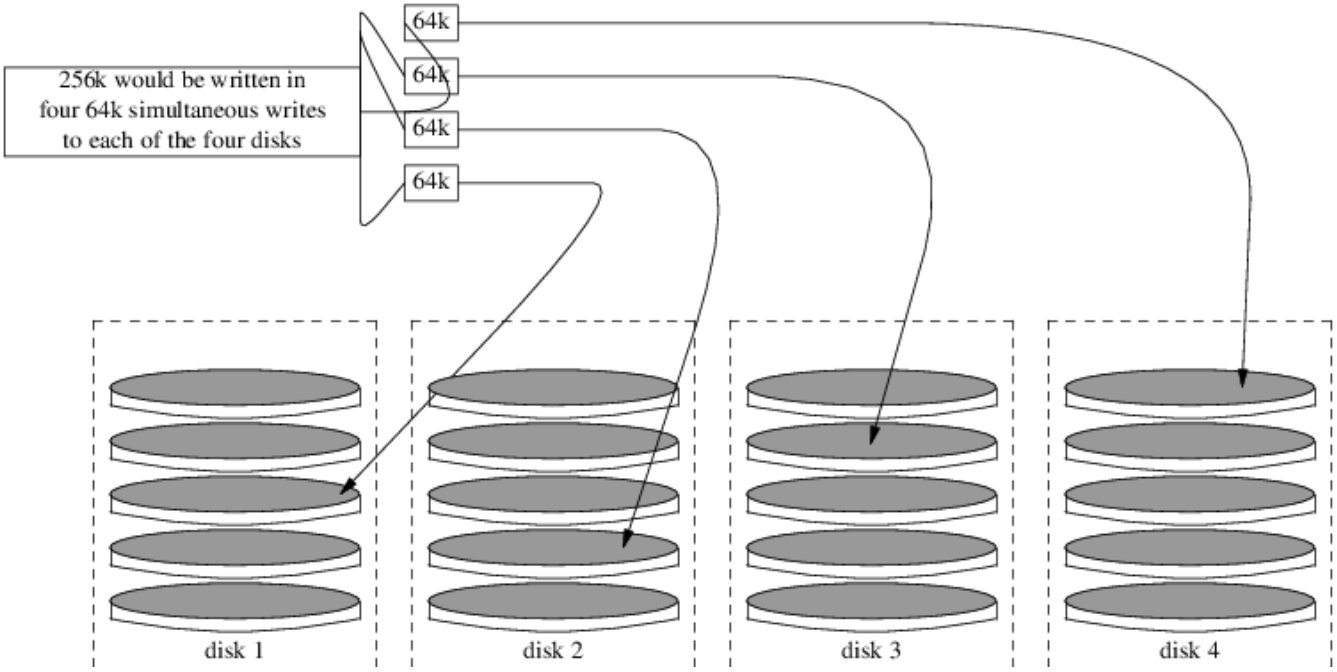
Before reading this chapter:

- Understand how FreeBSD treats disk devices (Storage).
- Know how to configure and install a new kernel (Configuring the FreeBSD Kernel).

21.2. RAID0 - Striping

Striping combines several disk drives into a single volume. Striping can be performed through the use of hardware RAID controllers. The GEOM disk subsystem provides software support for disk striping, also known as RAID0, without the need for a RAID disk controller.

In RAID0, data is split into blocks that are written across all the drives in the array. As seen in the following illustration, instead of having to wait on the system to write 256k to one disk, RAID0 can simultaneously write 64k to each of the four disks in the array, offering superior I/O performance. This performance can be enhanced further by using multiple disk controllers.



Each disk in a RAID0 stripe must be of the same size, since I/O requests are interleaved to read or write to multiple disks in parallel.



RAID0 does *not* provide any redundancy. This means that if one disk in the array fails, all of the data on the disks is lost. If the data is important, implement a backup strategy that regularly saves backups to a remote system or device.

The process for creating a software, GEOM-based RAID0 on a FreeBSD system using commodity disks is as follows. Once the stripe is created, refer to gstripe(8) for more information on how to control an existing stripe.

Procedure: Creating a Stripe of Unformatted ATA Disks

1. Load the geom_stripe.ko module:

```
# kldload geom_stripe
```

2. Ensure that a suitable mount point exists. If this volume will become a root partition, then temporarily use another mount point such as /mnt.
3. Determine the device names for the disks which will be striped, and create the new stripe device. For example, to stripe two unused and unpartitioned ATA disks with device names of /dev/ad2 and /dev/ad3:

```
# gstripe label -v st0 /dev/ad2 /dev/ad3
Metadata value stored on /dev/ad2.
Metadata value stored on /dev/ad3.
Done.
```

4. Write a standard label, also known as a partition table, on the new volume and install the

default bootstrap code:

```
# bsdlabel -wB /dev/stripe/st0
```

5. This process should create two other devices in /dev/stripe in addition to st0. Those include st0a and st0c. At this point, a UFS file system can be created on st0a using `newfs`:

   ```
   # newfs -U /dev/stripe/st0a
   ```

   Many numbers will glide across the screen, and after a few seconds, the process will be complete. The volume has been created and is ready to be mounted.

6. To manually mount the created disk stripe:

   ```
   # mount /dev/stripe/st0a /mnt
   ```

7. To mount this striped file system automatically during the boot process, place the volume information in /etc/fstab. In this example, a permanent mount point, named stripe, is created:

   ```
   # mkdir /stripe
   # echo "/dev/stripe/st0a /stripe ufs rw 2 2" \
   >> /etc/fstab
   ```

8. The geom_stripe.ko module must also be automatically loaded during system initialization, by adding a line to /boot/loader.conf:

   ```
   # echo 'geom_stripe_load="YES"' >> /boot/loader.conf
   ```

21.3. RAID1 - Mirroring

RAID1, or *mirroring*, is the technique of writing the same data to more than one disk drive. Mirrors are usually used to guard against data loss due to drive failure. Each drive in a mirror contains an identical copy of the data. When an individual drive fails, the mirror continues to work, providing data from the drives that are still functioning. The computer keeps running, and the administrator has time to replace the failed drive without user interruption.

Two common situations are illustrated in these examples. The first creates a mirror out of two new drives and uses it as a replacement for an existing single drive. The second example creates a mirror on a single new drive, copies the old drive's data to it, then inserts the old drive into the mirror. While this procedure is slightly more complicated, it only requires one new drive.

Traditionally, the two drives in a mirror are identical in model and capacity, but gmirror(8) does

not require that. Mirrors created with dissimilar drives will have a capacity equal to that of the smallest drive in the mirror. Extra space on larger drives will be unused. Drives inserted into the mirror later must have at least as much capacity as the smallest drive already in the mirror.



The mirroring procedures shown here are non-destructive, but as with any major disk operation, make a full backup first.



While dump(8) is used in these procedures to copy file systems, it does not work on file systems with soft updates journaling. See tunefs(8) for information on detecting and disabling soft updates journaling.

21.3.1. Metadata Issues

Many disk systems store metadata at the end of each disk. Old metadata should be erased before reusing the disk for a mirror. Most problems are caused by two particular types of leftover metadata: GPT partition tables and old metadata from a previous mirror.

GPT metadata can be erased with gpart(8). This example erases both primary and backup GPT partition tables from disk ada8:

```
# gpart destroy -F ada8
```

A disk can be removed from an active mirror and the metadata erased in one step using gmirror(8). Here, the example disk ada8 is removed from the active mirror gm4:

```
# gmirror remove gm4 ada8
```

If the mirror is not running, but old mirror metadata is still on the disk, use `gmirror clear` to remove it:

```
# gmirror clear ada8
```

gmirror(8) stores one block of metadata at the end of the disk. As GPT partition schemes also store metadata at the end of the disk, mirroring entire GPT disks with gmirror(8) is not recommended. MBR partitioning is used here because it only stores a partition table at the start of the disk and does not conflict with the mirror metadata.

21.3.2. Creating a Mirror with Two New Disks

In this example, FreeBSD has already been installed on a single disk, ada0. Two new disks, ada1 and ada2, have been connected to the system. A new mirror will be created on these two disks and used to replace the old single disk.

The geom_mirror.ko kernel module must either be built into the kernel or loaded at boot- or run-time. Manually load the kernel module now:

```
# gmirror load
```

Create the mirror with the two new drives:

```
# gmirror label -v gm0 /dev/ada1 /dev/ada2
```

gm0 is a user-chosen device name assigned to the new mirror. After the mirror has been started, this device name appears in /dev/mirror/.

MBR and bsdlabel partition tables can now be created on the mirror with gpart(8). This example uses a traditional file system layout, with partitions for /, swap, /var, /tmp, and /usr. A single / and a swap partition will also work.

Partitions on the mirror do not have to be the same size as those on the existing disk, but they must be large enough to hold all the data already present on ada0.

```
# gpart create -s MBR mirror/gm0
# gpart add -t freebsd -a 4k mirror/gm0
# gpart show mirror/gm0
=>       63  156301423  mirror/gm0  MBR  (74G)
         63         63                    - free -  (31k)
        126  156301299                1  freebsd  (74G)
  156301425         61                    - free -  (30k)
```

```
# gpart create -s BSD mirror/gm0s1
# gpart add -t freebsd-ufs  -a 4k -s 2g mirror/gm0s1
# gpart add -t freebsd-swap -a 4k -s 4g mirror/gm0s1
# gpart add -t freebsd-ufs  -a 4k -s 2g mirror/gm0s1
# gpart add -t freebsd-ufs  -a 4k -s 1g mirror/gm0s1
# gpart add -t freebsd-ufs  -a 4k mirror/gm0s1
# gpart show mirror/gm0s1
=>        0  156301299  mirror/gm0s1  BSD  (74G)
          0          2                     - free -  (1.0k)
          2    4194304                   1  freebsd-ufs  (2.0G)
    4194306    8388608                   2  freebsd-swap (4.0G)
   12582914    4194304                   4  freebsd-ufs  (2.0G)
   16777218    2097152                   5  freebsd-ufs  (1.0G)
   18874370  137426928                   6  freebsd-ufs  (65G)
  156301298          1                     - free -  (512B)
```

Make the mirror bootable by installing bootcode in the MBR and bsdlabel and setting the active slice:

```
# gpart bootcode -b /boot/mbr mirror/gm0
# gpart set -a active -i 1 mirror/gm0
```

```
# gpart bootcode -b /boot/boot mirror/gm0s1
```

Format the file systems on the new mirror, enabling soft-updates.

```
# newfs -U /dev/mirror/gm0s1a
# newfs -U /dev/mirror/gm0s1d
# newfs -U /dev/mirror/gm0s1e
# newfs -U /dev/mirror/gm0s1f
```

File systems from the original ada0 disk can now be copied onto the mirror with dump(8) and restore(8).

```
# mount /dev/mirror/gm0s1a /mnt
# dump -C16 -b64 -0aL -f - / | (cd /mnt && restore -rf -)
# mount /dev/mirror/gm0s1d /mnt/var
# mount /dev/mirror/gm0s1e /mnt/tmp
# mount /dev/mirror/gm0s1f /mnt/usr
# dump -C16 -b64 -0aL -f - /var | (cd /mnt/var && restore -rf -)
# dump -C16 -b64 -0aL -f - /tmp | (cd /mnt/tmp && restore -rf -)
# dump -C16 -b64 -0aL -f - /usr | (cd /mnt/usr && restore -rf -)
```

Edit /mnt/etc/fstab to point to the new mirror file systems:

```
# Device        Mountpoint  FStype  Options Dump    Pass#
/dev/mirror/gm0s1a  /       ufs rw  1   1
/dev/mirror/gm0s1b  none        swap    sw  0   0
/dev/mirror/gm0s1d  /var        ufs rw  2   2
/dev/mirror/gm0s1e  /tmp        ufs rw  2   2
/dev/mirror/gm0s1f  /usr        ufs rw  2   2
```

If the geom_mirror.ko kernel module has not been built into the kernel, /mnt/boot/loader.conf is edited to load the module at boot:

```
geom_mirror_load="YES"
```

Reboot the system to test the new mirror and verify that all data has been copied. The BIOS will see the mirror as two individual drives rather than a mirror. Since the drives are identical, it does not matter which is selected to boot.

See Troubleshooting if there are problems booting. Powering down and disconnecting the original ada0 disk will allow it to be kept as an offline backup.

In use, the mirror will behave just like the original single drive.

21.3.3. Creating a Mirror with an Existing Drive

In this example, FreeBSD has already been installed on a single disk, ada0. A new disk, ada1, has been connected to the system. A one-disk mirror will be created on the new disk, the existing system copied onto it, and then the old disk will be inserted into the mirror. This slightly complex procedure is required because `gmirror` needs to put a 512-byte block of metadata at the end of each disk, and the existing ada0 has usually had all of its space already allocated.

Load the geom_mirror.ko kernel module:

```
# gmirror load
```

Check the media size of the original disk with `diskinfo`:

```
# diskinfo -v ada0 | head -n3
/dev/ada0
        512             # sectorsize
        1000204821504   # mediasize in bytes (931G)
```

Create a mirror on the new disk. To make certain that the mirror capacity is not any larger than the original ada0 drive, gnop(8) is used to create a fake drive of the same size. This drive does not store any data, but is used only to limit the size of the mirror. When gmirror(8) creates the mirror, it will restrict the capacity to the size of gzero.nop, even if the new ada1 drive has more space. Note that the *1000204821504* in the second line is equal to ada0's media size as shown by `diskinfo` above.

```
# geom zero load
# gnop create -s 1000204821504 gzero
# gmirror label -v gm0 gzero.nop ada1
# gmirror forget gm0
```

Since gzero.nop does not store any data, the mirror does not see it as connected. The mirror is told to "forget" unconnected components, removing references to gzero.nop. The result is a mirror device containing only a single disk, ada1.

After creating gm0, view the partition table on ada0. This output is from a 1 TB drive. If there is some unallocated space at the end of the drive, the contents may be copied directly from ada0 to the new mirror.

However, if the output shows that all of the space on the disk is allocated, as in the following listing, there is no space available for the 512-byte mirror metadata at the end of the disk.

```
# gpart show ada0
=>        63  1953525105        ada0  MBR  (931G)
          63  1953525105           1  freebsd  [active]  (931G)
```

In this case, the partition table must be edited to reduce the capacity by one sector on mirror/gm0.

The procedure will be explained later.

In either case, partition tables on the primary disk should be first copied using `gpart backup` and `gpart restore`.

```
# gpart backup ada0 > table.ada0
# gpart backup ada0s1 > table.ada0s1
```

These commands create two files, table.ada0 and table.ada0s1. This example is from a 1 TB drive:

```
# cat table.ada0
MBR 4
1 freebsd         63 1953525105   [active]
```

```
# cat table.ada0s1
BSD 8
1  freebsd-ufs          0    4194304
2 freebsd-swap    4194304   33554432
4  freebsd-ufs   37748736   50331648
5  freebsd-ufs   88080384   41943040
6  freebsd-ufs  130023424  838860800
7  freebsd-ufs  968884224  984640881
```

If no free space is shown at the end of the disk, the size of both the slice and the last partition must be reduced by one sector. Edit the two files, reducing the size of both the slice and last partition by one. These are the last numbers in each listing.

```
# cat table.ada0
MBR 4
1 freebsd         63 1953525104   [active]
```

```
# cat table.ada0s1
BSD 8
1  freebsd-ufs          0    4194304
2 freebsd-swap    4194304   33554432
4  freebsd-ufs   37748736   50331648
5  freebsd-ufs   88080384   41943040
6  freebsd-ufs  130023424  838860800
7  freebsd-ufs  968884224  984640880
```

If at least one sector was unallocated at the end of the disk, these two files can be used without modification.

Now restore the partition table into mirror/gm0:

```
# gpart restore mirror/gm0 < table.ada0
# gpart restore mirror/gm0s1 < table.ada0s1
```

Check the partition table with `gpart show`. This example has gm0s1a for /, gm0s1d for /var, gm0s1e for /usr, gm0s1f for /data1, and gm0s1g for /data2.

```
# gpart show mirror/gm0
=>        63  1953525104  mirror/gm0  MBR  (931G)
          63  1953525042           1  freebsd  [active]  (931G)
  1953525105          62              - free -  (31k)

# gpart show mirror/gm0s1
=>         0  1953525042  mirror/gm0s1  BSD  (931G)
           0     2097152             1  freebsd-ufs  (1.0G)
     2097152    16777216             2  freebsd-swap  (8.0G)
    18874368    41943040             4  freebsd-ufs  (20G)
    60817408    20971520             5  freebsd-ufs  (10G)
    81788928   629145600             6  freebsd-ufs  (300G)
   710934528  1242590514             7  freebsd-ufs  (592G)
  1953525042          63                - free -  (31k)
```

Both the slice and the last partition must have at least one free block at the end of the disk.

Create file systems on these new partitions. The number of partitions will vary to match the original disk, ada0.

```
# newfs -U /dev/mirror/gm0s1a
# newfs -U /dev/mirror/gm0s1d
# newfs -U /dev/mirror/gm0s1e
# newfs -U /dev/mirror/gm0s1f
# newfs -U /dev/mirror/gm0s1g
```

Make the mirror bootable by installing bootcode in the MBR and bsdlabel and setting the active slice:

```
# gpart bootcode -b /boot/mbr mirror/gm0
# gpart set -a active -i 1 mirror/gm0
# gpart bootcode -b /boot/boot mirror/gm0s1
```

Adjust /etc/fstab to use the new partitions on the mirror. Back up this file first by copying it to /etc/fstab.orig.

```
# cp /etc/fstab /etc/fstab.orig
```

Edit /etc/fstab, replacing /dev/ada0 with mirror/gm0.

```
# Device        Mountpoint  FStype  Options Dump    Pass#
/dev/mirror/gm0s1a  /       ufs rw  1   1
/dev/mirror/gm0s1b  none        swap    sw  0   0
/dev/mirror/gm0s1d  /var        ufs rw  2   2
/dev/mirror/gm0s1e  /usr        ufs rw  2   2
/dev/mirror/gm0s1f  /data1      ufs rw  2   2
/dev/mirror/gm0s1g  /data2      ufs rw  2   2
```

If the geom_mirror.ko kernel module has not been built into the kernel, edit /boot/loader.conf to load it at boot:

```
geom_mirror_load="YES"
```

File systems from the original disk can now be copied onto the mirror with dump(8) and restore(8). Each file system dumped with `dump -L` will create a snapshot first, which can take some time.

```
# mount /dev/mirror/gm0s1a /mnt
# dump -C16 -b64 -0aL -f - /    | (cd /mnt && restore -rf -)
# mount /dev/mirror/gm0s1d /mnt/var
# mount /dev/mirror/gm0s1e /mnt/usr
# mount /dev/mirror/gm0s1f /mnt/data1
# mount /dev/mirror/gm0s1g /mnt/data2
# dump -C16 -b64 -0aL -f - /usr | (cd /mnt/usr && restore -rf -)
# dump -C16 -b64 -0aL -f - /var | (cd /mnt/var && restore -rf -)
# dump -C16 -b64 -0aL -f - /data1 | (cd /mnt/data1 && restore -rf -)
# dump -C16 -b64 -0aL -f - /data2 | (cd /mnt/data2 && restore -rf -)
```

Restart the system, booting from ada1. If everything is working, the system will boot from mirror/gm0, which now contains the same data as ada0 had previously. See Troubleshooting if there are problems booting.

At this point, the mirror still consists of only the single ada1 disk.

After booting from mirror/gm0 successfully, the final step is inserting ada0 into the mirror.



When ada0 is inserted into the mirror, its former contents will be overwritten by data from the mirror. Make certain that mirror/gm0 has the same contents as ada0 before adding ada0 to the mirror. If the contents previously copied by dump(8) and restore(8) are not identical to what was on ada0, revert /etc/fstab to mount the file systems on ada0, reboot, and start the whole procedure again.

```
# gmirror insert gm0 ada0
GEOM_MIRROR: Device gm0: rebuilding provider ada0
```

Synchronization between the two disks will start immediately. Use `gmirror status` to view the

progress.

```
# gmirror status
      Name    Status  Components
mirror/gm0  DEGRADED  ada1 (ACTIVE)
                      ada0 (SYNCHRONIZING, 64%)
```

After a while, synchronization will finish.

```
GEOM_MIRROR: Device gm0: rebuilding provider ada0 finished.
# gmirror status
      Name    Status  Components
mirror/gm0  COMPLETE  ada1 (ACTIVE)
                      ada0 (ACTIVE)
```

mirror/gm0 now consists of the two disks ada0 and ada1, and the contents are automatically synchronized with each other. In use, mirror/gm0 will behave just like the original single drive.

21.3.4. Troubleshooting

If the system no longer boots, BIOS settings may have to be changed to boot from one of the new mirrored drives. Either mirror drive can be used for booting, as they contain identical data.

If the boot stops with this message, something is wrong with the mirror device:

```
Mounting from ufs:/dev/mirror/gm0s1a failed with error 19.

Loader variables:
  vfs.root.mountfrom=ufs:/dev/mirror/gm0s1a
  vfs.root.mountfrom.options=rw

Manual root filesystem specification:
  <fstype>:<device> [options]
      Mount <device> using filesystem <fstype>
      and with the specified (optional) option list.

    e.g. ufs:/dev/da0s1a
         zfs:tank
         cd9660:/dev/acd0 ro
           (which is equivalent to: mount -t cd9660 -o ro /dev/acd0 /)

  ?               List valid disk boot devices
  .               Yield 1 second (for background tasks)
  <empty line>    Abort manual input

mountroot>
```

Forgetting to load the geom_mirror.ko module in /boot/loader.conf can cause this problem. To fix it, boot from a FreeBSD installation media and choose `Shell` at the first prompt. Then load the mirror module and mount the mirror device:

```
# gmirror load
# mount /dev/mirror/gm0s1a /mnt
```

Edit /mnt/boot/loader.conf, adding a line to load the mirror module:

```
geom_mirror_load="YES"
```

Save the file and reboot.

Other problems that cause `error 19` require more effort to fix. Although the system should boot from ada0, another prompt to select a shell will appear if /etc/fstab is incorrect. Enter `ufs:/dev/ada0s1a` at the boot loader prompt and press `Enter`. Undo the edits in /etc/fstab then mount the file systems from the original disk (ada0) instead of the mirror. Reboot the system and try the procedure again.

```
Enter full pathname of shell or RETURN for /bin/sh:
# cp /etc/fstab.orig /etc/fstab
# shutdown -r now
```

21.3.5. Recovering from Disk Failure

The benefit of disk mirroring is that an individual disk can fail without causing the mirror to lose any data. In the above example, if ada0 fails, the mirror will continue to work, providing data from the remaining working drive, ada1.

To replace the failed drive, shut down the system and physically replace the failed drive with a new drive of equal or greater capacity. Manufacturers use somewhat arbitrary values when rating drives in gigabytes, and the only way to really be sure is to compare the total count of sectors shown by `diskinfo -v`. A drive with larger capacity than the mirror will work, although the extra space on the new drive will not be used.

After the computer is powered back up, the mirror will be running in a "degraded" mode with only one drive. The mirror is told to forget drives that are not currently connected:

```
# gmirror forget gm0
```

Any old metadata should be cleared from the replacement disk using the instructions in Metadata Issues. Then the replacement disk, ada4 for this example, is inserted into the mirror:

```
# gmirror insert gm0 /dev/ada4
```

Resynchronization begins when the new drive is inserted into the mirror. This process of copying mirror data to a new drive can take a while. Performance of the mirror will be greatly reduced during the copy, so inserting new drives is best done when there is low demand on the computer.

Progress can be monitored with `gmirror status`, which shows drives that are being synchronized and the percentage of completion. During resynchronization, the status will be `DEGRADED`, changing to `COMPLETE` when the process is finished.

21.4. RAID3 - Byte-level Striping with Dedicated Parity

RAID3 is a method used to combine several disk drives into a single volume with a dedicated parity disk. In a RAID3 system, data is split up into a number of bytes that are written across all the drives in the array except for one disk which acts as a dedicated parity disk. This means that disk reads from a RAID3 implementation access all disks in the array. Performance can be enhanced by using multiple disk controllers. The RAID3 array provides a fault tolerance of 1 drive, while providing a capacity of 1 - 1/n times the total capacity of all drives in the array, where n is the number of hard drives in the array. Such a configuration is mostly suitable for storing data of larger sizes such as multimedia files.

At least 3 physical hard drives are required to build a RAID3 array. Each disk must be of the same size, since I/O requests are interleaved to read or write to multiple disks in parallel. Also, due to the nature of RAID3, the number of drives must be equal to 3, 5, 9, 17, and so on, or 2^n + 1.

This section demonstrates how to create a software RAID3 on a FreeBSD system.



While it is theoretically possible to boot from a RAID3 array on FreeBSD, that configuration is uncommon and is not advised.

21.4.1. Creating a Dedicated RAID3 Array

In FreeBSD, support for RAID3 is implemented by the graid3(8) GEOM class. Creating a dedicated RAID3 array on FreeBSD requires the following steps.

1. First, load the geom_raid3.ko kernel module by issuing one of the following commands:

   ```
   # graid3 load
   ```

   or:

   ```
   # kldload geom_raid3
   ```

2. Ensure that a suitable mount point exists. This command creates a new directory to use as the mount point:

   ```
   # mkdir /multimedia
   ```

3. Determine the device names for the disks which will be added to the array, and create the new RAID3 device. The final device listed will act as the dedicated parity disk. This example uses three unpartitioned ATA drives: ada1 and ada2 for data, and ada3 for parity.

```
# graid3 label -v gr0 /dev/ada1 /dev/ada2 /dev/ada3
Metadata value stored on /dev/ada1.
Metadata value stored on /dev/ada2.
Metadata value stored on /dev/ada3.
Done.
```

4. Partition the newly created gr0 device and put a UFS file system on it:

```
# gpart create -s GPT /dev/raid3/gr0
# gpart add -t freebsd-ufs /dev/raid3/gr0
# newfs -j /dev/raid3/gr0p1
```

   Many numbers will glide across the screen, and after a bit of time, the process will be complete. The volume has been created and is ready to be mounted:

```
# mount /dev/raid3/gr0p1 /multimedia/
```

   The RAID3 array is now ready to use.

Additional configuration is needed to retain this setup across system reboots.

1. The geom_raid3.ko module must be loaded before the array can be mounted. To automatically load the kernel module during system initialization, add the following line to /boot/loader.conf:

```
geom_raid3_load="YES"
```

2. The following volume information must be added to /etc/fstab in order to automatically mount the array's file system during the system boot process:

```
/dev/raid3/gr0p1    /multimedia ufs rw  2   2
```

21.5. Software RAID Devices

Some motherboards and expansion cards add some simple hardware, usually just a ROM, that allows the computer to boot from a RAID array. After booting, access to the RAID array is handled by software running on the computer's main processor. This "hardware-assisted software RAID" gives RAID arrays that are not dependent on any particular operating system, and which are functional even before an operating system is loaded.

Several levels of RAID are supported, depending on the hardware in use. See graid(8) for a complete

list.

graid(8) requires the geom_raid.ko kernel module, which is included in the GENERIC kernel starting with FreeBSD 9.1. If needed, it can be loaded manually with `graid load`.

21.5.1. Creating an Array

Software RAID devices often have a menu that can be entered by pressing special keys when the computer is booting. The menu can be used to create and delete RAID arrays. graid(8) can also create arrays directly from the command line.

`graid label` is used to create a new array. The motherboard used for this example has an Intel software RAID chipset, so the Intel metadata format is specified. The new array is given a label of gm0, it is a mirror (RAID1), and uses drives ada0 and ada1.



Some space on the drives will be overwritten when they are made into a new array. Back up existing data first!

```
# graid label Intel gm0 RAID1 ada0 ada1
GEOM_RAID: Intel-a29ea104: Array Intel-a29ea104 created.
GEOM_RAID: Intel-a29ea104: Disk ada0 state changed from NONE to ACTIVE.
GEOM_RAID: Intel-a29ea104: Subdisk gm0:0-ada0 state changed from NONE to ACTIVE.
GEOM_RAID: Intel-a29ea104: Disk ada1 state changed from NONE to ACTIVE.
GEOM_RAID: Intel-a29ea104: Subdisk gm0:1-ada1 state changed from NONE to ACTIVE.
GEOM_RAID: Intel-a29ea104: Array started.
GEOM_RAID: Intel-a29ea104: Volume gm0 state changed from STARTING to OPTIMAL.
Intel-a29ea104 created
GEOM_RAID: Intel-a29ea104: Provider raid/r0 for volume gm0 created.
```

A status check shows the new mirror is ready for use:

```
# graid status
   Name   Status  Components
raid/r0  OPTIMAL  ada0 (ACTIVE (ACTIVE))
                  ada1 (ACTIVE (ACTIVE))
```

The array device appears in /dev/raid/. The first array is called r0. Additional arrays, if present, will be r1, r2, and so on.

The BIOS menu on some of these devices can create arrays with special characters in their names. To avoid problems with those special characters, arrays are given simple numbered names like r0. To show the actual labels, like gm0 in the example above, use sysctl(8):

```
# sysctl kern.geom.raid.name_format=1
```

21.5.2. Multiple Volumes

Some software RAID devices support more than one *volume* on an array. Volumes work like partitions, allowing space on the physical drives to be split and used in different ways. For example, Intel software RAID devices support two volumes. This example creates a 40 G mirror for safely storing the operating system, followed by a 20 G RAID0 (stripe) volume for fast temporary storage:

```
# graid label -S 40G Intel gm0 RAID1 ada0 ada1
# graid add -S 20G gm0 RAID0
```

Volumes appear as additional rX entries in /dev/raid/. An array with two volumes will show r0 and r1.

See graid(8) for the number of volumes supported by different software RAID devices.

21.5.3. Converting a Single Drive to a Mirror

Under certain specific conditions, it is possible to convert an existing single drive to a graid(8) array without reformatting. To avoid data loss during the conversion, the existing drive must meet these minimum requirements:

- The drive must be partitioned with the MBR partitioning scheme. GPT or other partitioning schemes with metadata at the end of the drive will be overwritten and corrupted by the graid(8) metadata.
- There must be enough unpartitioned and unused space at the end of the drive to hold the graid(8) metadata. This metadata varies in size, but the largest occupies 64 M, so at least that much free space is recommended.

If the drive meets these requirements, start by making a full backup. Then create a single-drive mirror with that drive:

```
# graid label Intel gm0 RAID1 ada0 NONE
```

graid(8) metadata was written to the end of the drive in the unused space. A second drive can now be inserted into the mirror:

```
# graid insert raid/r0 ada1
```

Data from the original drive will immediately begin to be copied to the second drive. The mirror will operate in degraded status until the copy is complete.

21.5.4. Inserting New Drives into the Array

Drives can be inserted into an array as replacements for drives that have failed or are missing. If there are no failed or missing drives, the new drive becomes a spare. For example, inserting a new drive into a working two-drive mirror results in a two-drive mirror with one spare drive, not a

three-drive mirror.

In the example mirror array, data immediately begins to be copied to the newly-inserted drive. Any existing information on the new drive will be overwritten.

```
# graid insert raid/r0 ada1
GEOM_RAID: Intel-a29ea104: Disk ada1 state changed from NONE to ACTIVE.
GEOM_RAID: Intel-a29ea104: Subdisk gm0:1-ada1 state changed from NONE to NEW.
GEOM_RAID: Intel-a29ea104: Subdisk gm0:1-ada1 state changed from NEW to REBUILD.
GEOM_RAID: Intel-a29ea104: Subdisk gm0:1-ada1 rebuild start at 0.
```

21.5.5. Removing Drives from the Array

Individual drives can be permanently removed from a from an array and their metadata erased:

```
# graid remove raid/r0 ada1
GEOM_RAID: Intel-a29ea104: Disk ada1 state changed from ACTIVE to OFFLINE.
GEOM_RAID: Intel-a29ea104: Subdisk gm0:1-[unknown] state changed from ACTIVE to NONE.
GEOM_RAID: Intel-a29ea104: Volume gm0 state changed from OPTIMAL to DEGRADED.
```

21.5.6. Stopping the Array

An array can be stopped without removing metadata from the drives. The array will be restarted when the system is booted.

```
# graid stop raid/r0
```

21.5.7. Checking Array Status

Array status can be checked at any time. After a drive was added to the mirror in the example above, data is being copied from the original drive to the new drive:

```
# graid status
   Name    Status  Components
raid/r0  DEGRADED  ada0 (ACTIVE (ACTIVE))
                   ada1 (ACTIVE (REBUILD 28%))
```

Some types of arrays, like `RAID0` or `CONCAT`, may not be shown in the status report if disks have failed. To see these partially-failed arrays, add `-ga`:

```
# graid status -ga
          Name  Status  Components
Intel-e2d07d9a  BROKEN  ada6 (ACTIVE (ACTIVE))
```

21.5.8. Deleting Arrays

Arrays are destroyed by deleting all of the volumes from them. When the last volume present is deleted, the array is stopped and metadata is removed from the drives:

```
# graid delete raid/r0
```

21.5.9. Deleting Unexpected Arrays

Drives may unexpectedly contain graid(8) metadata, either from previous use or manufacturer testing. graid(8) will detect these drives and create an array, interfering with access to the individual drive. To remove the unwanted metadata:

1. Boot the system. At the boot menu, select 2 for the loader prompt. Enter:

   ```
   OK set kern.geom.raid.enable=0
   OK boot
   ```

   The system will boot with graid(8) disabled.

2. Back up all data on the affected drive.
3. As a workaround, graid(8) array detection can be disabled by adding

   ```
   kern.geom.raid.enable=0
   ```

   to /boot/loader.conf.

   To permanently remove the graid(8) metadata from the affected drive, boot a FreeBSD installation CD-ROM or memory stick, and select `Shell`. Use `status` to find the name of the array, typically `raid/r0`:

   ```
   # graid status
      Name   Status  Components
   raid/r0  OPTIMAL  ada0 (ACTIVE (ACTIVE))
                     ada1 (ACTIVE (ACTIVE))
   ```

   Delete the volume by name:

   ```
   # graid delete raid/r0
   ```

   If there is more than one volume shown, repeat the process for each volume. After the last array has been deleted, the volume will be destroyed.

   Reboot and verify data, restoring from backup if necessary. After the metadata has been

removed, the `kern.geom.raid.enable=0` entry in /boot/loader.conf can also be removed.

21.6. GEOM Gate Network

GEOM provides a simple mechanism for providing remote access to devices such as disks, CDs, and file systems through the use of the GEOM Gate network daemon, ggated. The system with the device runs the server daemon which handles requests made by clients using ggatec. The devices should not contain any sensitive data as the connection between the client and the server is not encrypted.

Similar to NFS, which is discussed in Network File System (NFS), ggated is configured using an exports file. This file specifies which systems are permitted to access the exported resources and what level of access they are offered. For example, to give the client `192.168.1.5` read and write access to the fourth slice on the first SCSI disk, create /etc/gg.exports with this line:

```
192.168.1.5 RW /dev/da0s4d
```

Before exporting the device, ensure it is not currently mounted. Then, start ggated:

```
# ggated
```

Several options are available for specifying an alternate listening port or changing the default location of the exports file. Refer to ggated(8) for details.

To access the exported device on the client machine, first use `ggatec` to specify the IP address of the server and the device name of the exported device. If successful, this command will display a `ggate` device name to mount. Mount that specified device name on a free mount point. This example connects to the /dev/da0s4d partition on `192.168.1.1`, then mounts /dev/ggate0 on /mnt:

```
# ggatec create -o rw 192.168.1.1 /dev/da0s4d
ggate0
# mount /dev/ggate0 /mnt
```

The device on the server may now be accessed through /mnt on the client. For more details about `ggatec` and a few usage examples, refer to ggatec(8).



The mount will fail if the device is currently mounted on either the server or any other client on the network. If simultaneous access is needed to network resources, use NFS instead.

When the device is no longer needed, unmount it with `umount` so that the resource is available to other clients.

21.7. Labeling Disk Devices

During system initialization, the FreeBSD kernel creates device nodes as devices are found. This

method of probing for devices raises some issues. For instance, what if a new disk device is added via USB? It is likely that a flash device may be handed the device name of da0 and the original da0 shifted to da1. This will cause issues mounting file systems if they are listed in /etc/fstab which may also prevent the system from booting.

One solution is to chain SCSI devices in order so a new device added to the SCSI card will be issued unused device numbers. But what about USB devices which may replace the primary SCSI disk? This happens because USB devices are usually probed before the SCSI card. One solution is to only insert these devices after the system has been booted. Another method is to use only a single ATA drive and never list the SCSI devices in /etc/fstab.

A better solution is to use `glabel` to label the disk devices and use the labels in /etc/fstab. Since `glabel` stores the label in the last sector of a given provider, the label will remain persistent across reboots. By using this label as a device, the file-system may always be mounted regardless of what device node it is accessed through.



`glabel` can create both transient and permanent labels. Only permanent labels are consistent across reboots. Refer to glabel(8) for more information on the differences between labels.

21.7.1. Label Types and Examples

Permanent labels can be a generic or a file system label. Permanent file system labels can be created with tunefs(8) or newfs(8). These types of labels are created in a sub-directory of /dev, and will be named according to the file system type. For example, UFS2 file system labels will be created in /dev/ufs. Generic permanent labels can be created with `glabel label`. These are not file system specific and will be created in /dev/label.

Temporary labels are destroyed at the next reboot. These labels are created in /dev/label and are suited to experimentation. A temporary label can be created using `glabel create`.

To create a permanent label for a UFS2 file system without destroying any data, issue the following command:

```
# tunefs -L home /dev/da3
```

A label should now exist in /dev/ufs which may be added to /etc/fstab:

```
/dev/ufs/home       /home           ufs     rw              2       2
```



The file system must not be mounted while attempting to run `tunefs`.

Now the file system may be mounted:

```
# mount /home
```